



Gopi Patel
Full-stack web development · Project report

Sugar Rush

A role-based online ordering platform for a dessert retailer

A full-stack web application that lets customers browse a themed dessert catalogue, build a cart without signing in, and place orders that automatically apply live promotions — while giving sales staff an order-fulfilment desk and administrators full control of users, inventory and reporting.

Next.js 14 · App Router

TypeScript (strict)

Prisma + PostgreSQL

NextAuth

Server Actions

Tailwind CSS

Vercel

SCALE	SURFACE	ROLES	STATUS
71 source files 5,502 lines	23 pages · 3 APIs 13 server actions	Admin · Sales Customer	Feature-complete core Deployment-ready

CONTENTS

Table of contents

1	Project overview	13	Implementation — sales
2	Objectives	14	Implementation — admin
3	Requirements	15	Challenges & solutions
4	Solution overview	16	Testing & verification
5	Features & capabilities	17	Security & data integrity
6	System architecture	18	Deployment & usage
7	Technology stack	19	Role & contribution
8	Core workflows	20	Results & verified metrics
9	Engineering decisions	21	Limitations
10	AI-assisted development	22	Future improvements
11	Implementation — storefront	23	Conclusion
12	Implementation — customer	24	Master Prompt

About the evidence in this report

Every figure, screenshot and metric in this document was taken from the project itself rather than from prior documentation. Screenshots were captured from the application running against its live PostgreSQL database at a 1440×900 viewport; the order totals, stock levels and charts shown are real records. Build and type-check results in section 16 are the actual output of `next build` and `tsc --noEmit` . Where a feature is incomplete it is labelled as such, and section 21 lists known limitations in full.

Credentials

No account passwords, database connection strings, API tokens or other secrets appear anywhere in this report. Demo account *addresses* are referenced where they aid understanding; their passwords are not reproduced.

SECTION 1

Project overview

Sugar Rush is a web application for a dessert shop that sells ice cream, thick shakes, smoothies, donuts and baked desserts. It covers the complete path from a visitor discovering a product to staff marking that order delivered and issuing an invoice.

71

SOURCE FILES

5,502

LINES OF CODE

27

BUILD ROUTES

13

SERVER ACTIONS

5

DATA MODELS

What the system does

The application serves three distinct audiences from one codebase, each with its own entry point, navigation and permission boundary:

AUDIENCE	ENTRY POINT	WHAT THEY CAN DO
Visitors	Public storefront at <code>/</code>	Browse the full catalogue, filter within a product category, and add items to a cart — all without creating an account. An account is only required at checkout.
Customers	<code>/customer/*</code>	Place orders, track and cancel them, view and print invoices, raise support requests, and manage their own account details including a birthday used by one of the promotions.
Sales staff	<code>/sales/*</code>	Work the fulfilment queue: view all orders, move them between states, look up a customer's history, issue invoices, and check stock levels read-only.
Administrators	<code>/admin/*</code>	Manage user accounts and role assignment, maintain the product catalogue including image uploads, export inventory to CSV, and review monthly and yearly sales reporting.

What distinguishes it technically

- **A promotions engine that the client cannot influence.** Three live offers are computed by one shared module. The browser uses it to preview a saving; the server re-runs it from its own clock and its own price list when the order is created. Section 9 covers the design.
- **Authorisation applied in three independent layers** — route middleware, a per-action session guard, and ownership checks on individual records — so no single bypass is sufficient.
- **A themed storefront driven entirely by configuration.** Four product "worlds" each carry their own palette, background, particle set and filter rules in one config file; selecting a world re-themes the entire page beneath the hero.
- **Catalogue imagery sourced by an automated, licence-aware pipeline** that verifies the dominant colour of each candidate photograph and records attribution (section 11).

SECTION 2

Objectives

#	OBJECTIVE	WHY IT MATTERED & HOW IT WAS MET
1	Support three operational roles from one application	A shop needs shopping, fulfilment and administration to be separate concerns without three separate deployments. Met with role-scoped route groups, each with its own layout and its own server-action module.
2	Let people shop before they commit to an account	Requiring registration before browsing loses casual visitors. Met with a cart persisted in <code>localStorage</code> that survives the login round-trip, so only checkout requires signing in.
3	Run real promotions without creating a discount loophole	Offers advertised on a storefront have to be enforced server-side or they become an attack surface. Met with a single shared calculation module whose security-relevant inputs come only from the server.
4	Preserve the integrity of historical records	Invoices must stay correct after prices change, products are retired, or a customer renames themselves. Met by snapshotting the customer name and unit price onto each order and refusing deletes that would break references.
5	Deploy on managed, no-maintenance infrastructure	There should be no server to patch or keep running. Met by targeting a serverless host with pooled database connections, a build that needs no database, and object storage for uploads.
6	Present the catalogue attractively enough to drive browsing	A dessert shop competes visually. Met with a video-led, theme-switching storefront that stays responsive and respects reduced-motion preferences.

SECTION 3

Requirements

The requirements below are the ones the delivered system satisfies. Each is traceable to code in the repository; the right-hand column names where it is implemented.

Functional requirements

REF	REQUIREMENT	IMPLEMENTED IN
FR-1	Visitors can browse the catalogue and build a cart anonymously	<code>components/CartContext.tsx</code> , <code>app/page.tsx</code>
FR-2	Customers can self-register; staff accounts are created by an admin only	<code>app/api/signup/route.ts</code> , <code>app/admin/actions.ts</code>
FR-3	A single login routes each user to the area matching their role	<code>app/login/page.tsx</code> , <code>lib/auth.ts</code>
FR-4	Customers place orders with a delivery address and payment method	<code>placeOrder</code> in <code>app/customer/actions.ts</code>
FR-5	Eligible promotions apply automatically at order time	<code>lib/offers.ts</code>
FR-6	Stock decrements atomically with order creation	<code>prisma.\$transaction</code> in <code>placeOrder</code>
FR-7	Customers can cancel their own orders while still pending	<code>cancelOrder</code>
FR-8	Customers can view and print invoices for their own orders only	<code>app/customer/invoices/[id]/page.tsx</code>
FR-9	Customers can submit support requests	<code>submitSupportRequest</code>
FR-10	Customers can update their profile, password and birthday	<code>updateAccount</code>
FR-11	Sales staff can view all orders and change order status	<code>updateOrderStatus</code> in <code>app/sales/actions.ts</code>
FR-12	Sales staff can view customer directories and per-customer history	<code>app/sales/customers/</code>
FR-13	Sales staff can view stock but not modify it	<code>app/sales/inventory/page.tsx</code> (no write action exists)
FR-14	Admins have full CRUD over users with role assignment	<code>createUser</code> / <code>updateUser</code> / <code>deleteUser</code>
FR-15	Admins have full CRUD over products including image upload	<code>createProduct</code> / <code>updateProduct</code> / <code>deleteProduct</code> , <code>lib/blob.ts</code>
FR-16	Admins can export the inventory as CSV	<code>app/api/admin/export-inventory/route.ts</code>
FR-17	Admins can review monthly and yearly sales	<code>app/admin/sales-report/page.tsx</code> , <code>components/SalesCharts.tsx</code>

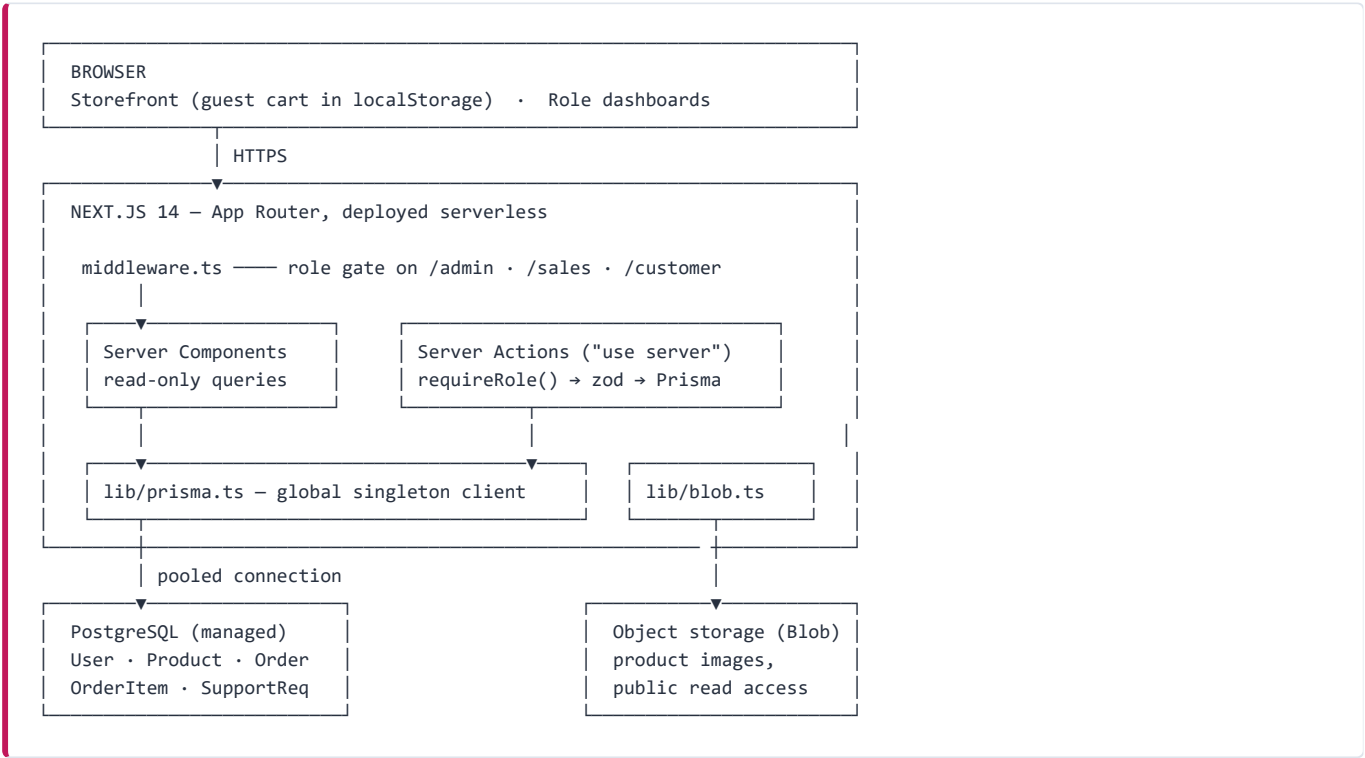
Non-functional requirements

REF	REQUIREMENT	HOW IT IS SATISFIED
NFR-1	Type safety	TypeScript <code>strict</code> mode across all 71 files; verified clean (section 16).
NFR-2	Input validation	8 zod schemas cover every server action and API route input, each with explicit bounds.
NFR-3	Authorisation	Three independent layers; see section 17.
NFR-4	Monetary accuracy	All money stored as <code>Decimal(10,2)</code> ; no floating-point currency anywhere.
NFR-5	Serverless compatibility	Global Prisma singleton, pooled connection, no build-time database dependency.
NFR-6	Graceful degradation	Storefront renders without a database; optional gifts skip when out of stock; asset cleanup failures never fail a request.
NFR-7	Responsive & accessible motion	Portrait/landscape-specific media handling; <code>prefers-reduced-motion</code> honoured by the scroll driver and particle layers.
NFR-8	Asset licensing	Catalogue imagery restricted to permissive licences with attribution recorded in <code>public/products/CREDITS.json</code> .

SECTION 4

Solution overview

A single Next.js application deployed as serverless functions, backed by managed PostgreSQL and object storage. There is no separate API tier, no long-running process, and no server to administer.



Design principles the implementation follows

PRINCIPLE	APPLIED AS
Server owns the truth	Prices, totals, discounts, roles and ownership are always re-derived server-side. The client submits identifiers and quantities, never values.
Guard beside the mutation	Every role folder's <code>actions.ts</code> opens with its own <code>require<Role>()</code> helper, so a new action makes the missing guard obvious in review.
Configuration over duplication	Branding, dessert-world theming, hero content and offer parameters each live in exactly one module and are consumed everywhere else.
Fail soft on the inessential	A promotional gift that is out of stock is skipped rather than blocking the order; failed image cleanup is swallowed; a database outage degrades the storefront to a brochure page.
Fail loud on the essential	A misconfigured image store produces an explicit, actionable error rather than a silent upload that loses the file.

SECTION 5

Features & capabilities

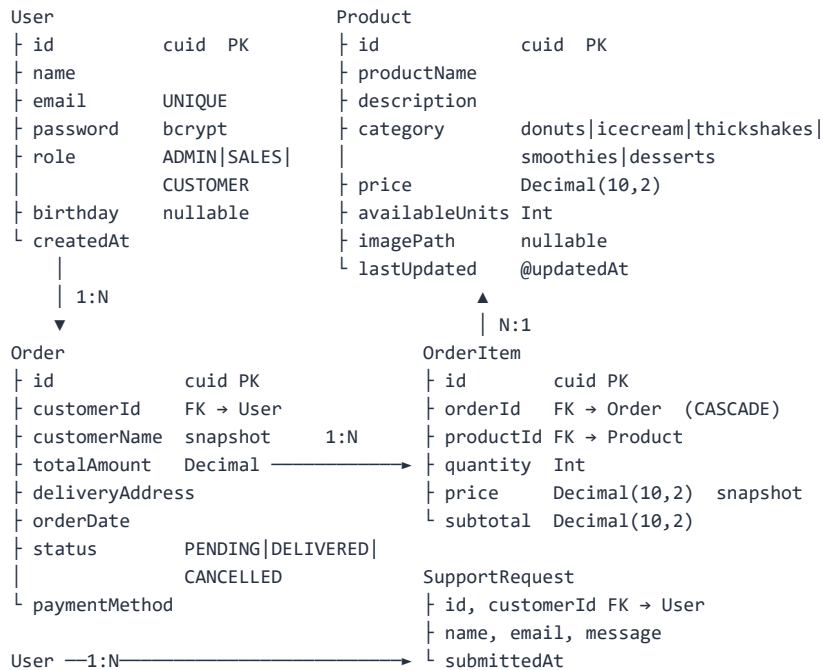
Status legend — **Complete**: implemented and exercised against live data. **Partial**: works but with a documented gap. **Stub**: visible in the UI, deliberately disabled.

CAPABILITY	STATUS	NOTES
Themed storefront with video hero	Complete	Four dessert worlds, 8-second looping video per world, full page re-theming on selection.
Anonymous browsing & cart	Complete	Persisted in <code>localStorage</code> ; survives refresh and the login round-trip.
Category filter tags	Complete	Derived at render time from regex rules per world — no database column required.
Customer registration & login	Complete	Role fixed server-side at signup; single login form for all roles.
Checkout with address & payment method	Partial	Captures intent (Cash on Delivery / UPI / Card). No payment is actually processed — see section 21.
Promotions engine	Complete	Three offers, server-authoritative. Verified against live orders in section 16.
Stock management	Partial	Decrements transactionally at order time. Cancellation does not restock — see section 21.
Order history & cancellation	Complete	Ownership- and state-checked before cancelling.
Order tracking	Stub	A disabled "Track your order" control is shown with a "Coming soon" label so the intent is visible without implying it works.
Printable invoices	Complete	Customer-scoped and staff-scoped variants; print styles hide navigation chrome.
Support requests	Partial	Submitted and stored, but no staff-facing inbox reads them — see section 21.
Order status workflow	Complete	Sales-only transition between Pending / Delivered / Cancelled, with dependent pages revalidated.
User administration	Complete	Create, edit, delete, assign roles; self-deletion blocked.
Product administration	Complete	Full CRUD with validated image upload and referential-safe deletion.
CSV inventory export	Complete	Admin-only, self-guarded, with correct CSV escaping.
Sales reporting	Complete	Monthly bar chart, three-year line chart, full order history table.
Password reset	Not built	No self-service recovery exists — see section 21.
Email notifications	Not built	No transactional email of any kind.

SECTION 6

System architecture

Data model



Why two fields are denormalised

`Order.customerName` and `OrderItem.price` duplicate data that exists elsewhere. Both are deliberate snapshots: an invoice issued in August must still show the name and the unit price that applied in August, even after the customer renames themselves or an admin re-prices the product. Without them, historical invoices would silently rewrite themselves.

Request paths

PATH	SEQUENCE
Protected page	<code>middleware</code> checks JWT presence and role-vs-prefix → Server Component queries Prisma → HTML streamed. A role mismatch redirects to that user's own dashboard rather than showing an error.
Mutation	Client invokes a Server Action → <code>require<Role>()</code> re-reads the session → zod parses the payload → Prisma writes → <code>revalidatePath()</code> refreshes every affected page.
Public storefront	Server Component fetches the catalogue inside a <code>try/catch</code> , sets <code>revalidate = 60</code> for ISR caching, and hands a plain serialised array to the client experience. Admin product writes call <code>revalidatePath("/")</code> to invalidate it immediately.
CSV export	Sits outside the middleware matcher, so the route handler performs its own session check and returns 403 for any non-admin.

Code organisation

AREA	FILES	LINES	RESPONSIBILITY
app/	34	1,765	Routes, layouts, server actions, API handlers
components/	25	2,910	Shared UI and the storefront experience
lib/	8	613	Prisma client, auth, uploads, offers, theming, formatting, branding
prisma/	1	102	Seed script (schema is .prisma)
scripts/	1	48	Admin password rotation utility
types/	1	25	NextAuth session/JWT augmentation
middleware.ts	1	39	Role-based route protection
Total	71	5,502	

SECTION 7

Technology stack

Runtime dependencies and their specific role

PACKAGE	VERSION	USED FOR
<code>next</code>	14.2.15	App Router, Server Components, Server Actions, middleware, image optimisation.
<code>react</code> / <code>react-dom</code>	^18.3.1	UI runtime.
<code>@prisma/client</code>	^5.20.0	All database access. No raw SQL is used anywhere.
<code>next-auth</code>	^4.24.8	Credentials authentication, JWT sessions with an 8-hour lifetime, role claim, route middleware.
<code>bcryptjs</code>	^2.4.3	Password hashing at 10 salt rounds on signup, admin user writes and self-service account updates.
<code>zod</code>	^3.23.8	Input validation on all 13 server actions and the signup endpoint.
<code>@vercel/blob</code>	^0.27.0	Product image storage with public read access.
<code>recharts</code>	^2.12.7	Monthly bar chart and yearly line chart in the sales report.
<code>framer-motion</code>	^12.42.2	Hero transitions, world morphing, cart drawer, particle motion with reduced-motion support.
<code>gsap</code>	^3.15.0	Homepage timeline animation.
<code>lenis</code>	^1.3.25	Smooth-scroll driver for the single-page storefront.

Tooling

PACKAGE	VERSION	USED FOR
<code>typescript</code>	^5.6.2	Strict-mode type checking across the whole project.
<code>prisma</code>	^5.20.0	Schema, client generation, <code>db push</code> , Studio.
<code>tailwindcss</code>	^3.4.13	Utility CSS plus a project-specific token set and component layer.
<code>tsx</code>	^4.19.1	Runs the TypeScript seed and maintenance scripts directly.
<code>postcss</code> , <code>autoprefixer</code>	—	Tailwind build pipeline.

Scripts

COMMAND	EFFECT
<code>npm run dev</code>	Local development server.
<code>npm run build</code>	<code>prisma generate && next build</code> — generation first so the client exists at compile time.

COMMAND	EFFECT
<code>postinstall</code>	<code>prisma generate</code> — makes a hosted build work with no extra configuration.
<code>npm run db:push</code> / <code>db:seed</code> / <code>db:studio</code>	Schema push, idempotent seeding, database GUI.

Configuration surface

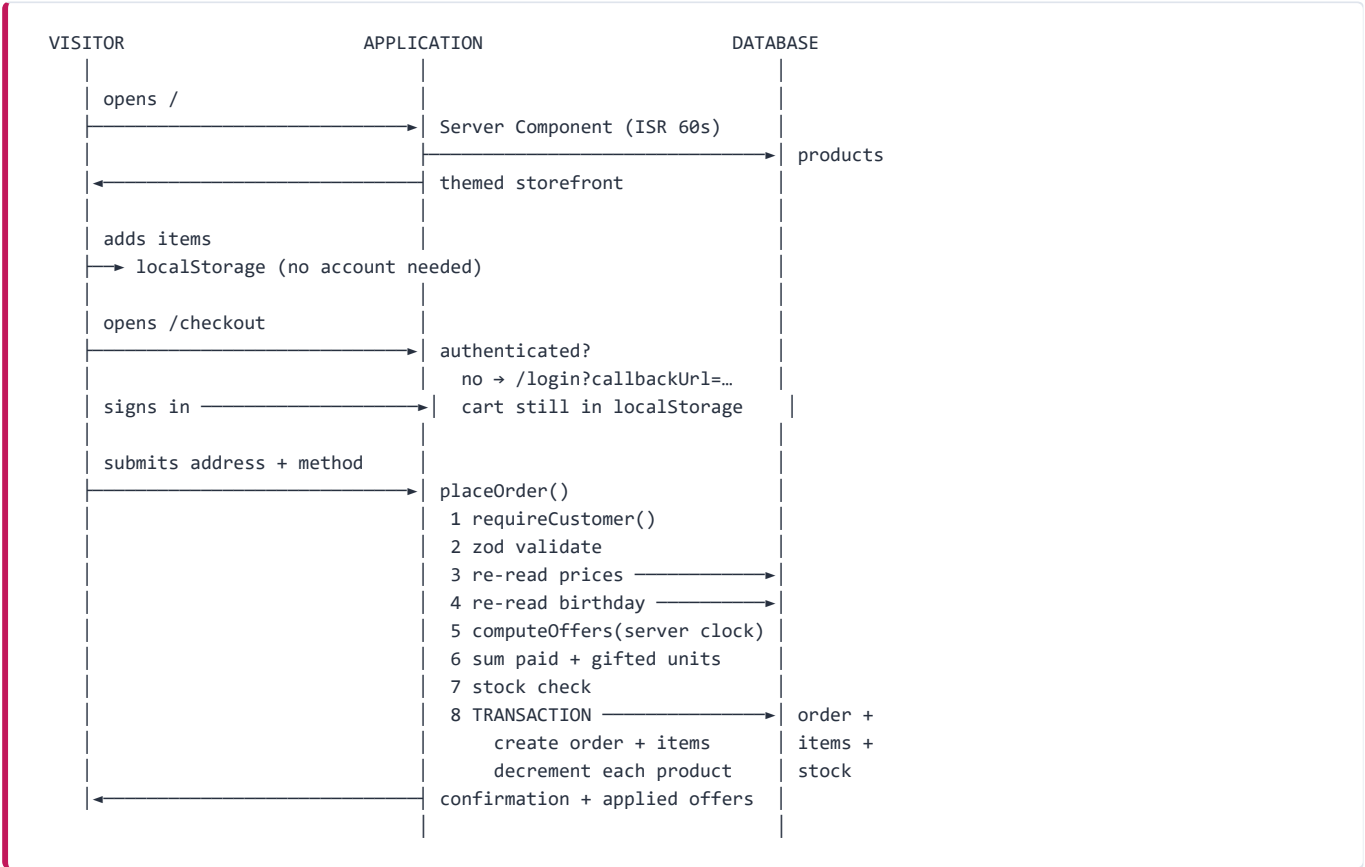
VARIABLE	PURPOSE
<code>DATABASE_URL</code>	Runtime connection. Must be the pooled endpoint in a serverless deployment to avoid exhausting connections.
<code>DIRECT_URL</code>	Non-pooled connection used for schema push and migrations.
<code>NEXTAUTH_SECRET</code>	JWT signing secret.
<code>NEXTAUTH_URL</code>	Canonical application URL; auto-derived from the host platform when unset (section 9).
<code>BLOB_READ_WRITE_TOKEN</code>	Object-store token. The store must be created with public read access.

All five are supplied through environment variables. `.env` is git-ignored; a committed `.env.example` documents the shape without any values.

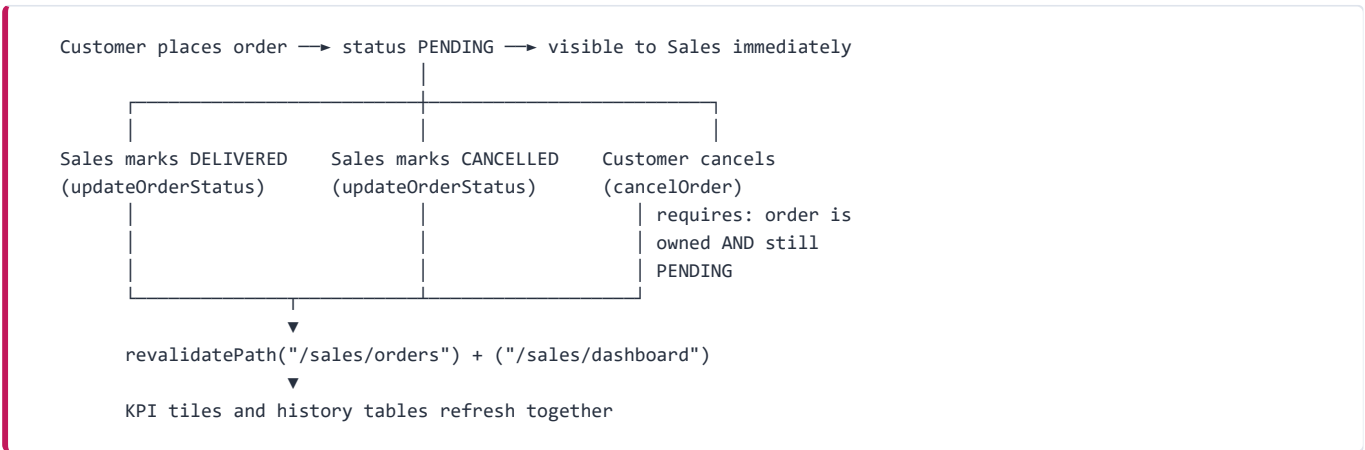
SECTION 8

Core workflows

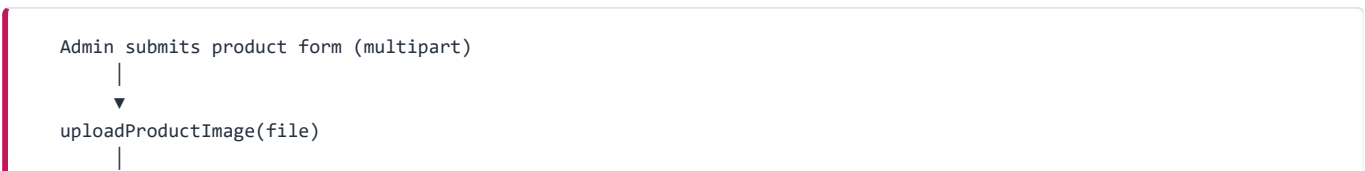
Workflow 1 — Guest browse to completed order



Workflow 2 — Order fulfilment



Workflow 3 — Product image upload



```
| reject unless MIME ∈ {jpeg, png, webp, gif}
| reject if size > 5 MB
| filename := -<4 random bytes as hex>.    ← never the user's
|
| token present? → Blob.put(access:"public") → return CDN url
|               | throws (e.g. private store)
|               ▼
|               → write to public/products/ → return "/products/..."
|               | throws (read-only host)
|               ▼
|               explicit error naming cause + fix
```

SECTION 9

Important engineering decisions

9.1 — Promotions computed by one module, trusted only from the server

PROBLEM	Three offers are advertised on the storefront: a donut multi-buy, a time-limited shake discount, and a birthday gift. The cart must preview the saving live, but any value the browser computes is attacker-controlled. Duplicating the rules in client and server code would also let the two drift apart.
DECISION	Write the rules once as a pure function , <code>computeOffers(lines, ctx)</code> , and make the security-relevant half of its input — <code>ctx</code> — obtainable only from the server.
IMPLEMENTATION	The cart calls a server action, <code>getOfferContext()</code> , to learn whether it is currently happy hour and whether it is the user's birthday, then runs the same function locally for display. <code>placeOrder</code> ignores every number the client sent, re-reads each product's price and category from the database, re-reads the birthday from the user row, evaluates its own clock, and recomputes. Time is resolved with <code>Intl.DateTimeFormat</code> pinned to the store's timezone, so the shopper's device clock and locale are irrelevant.
RESULT	A single definition of each offer, a live preview, and a stored total that is always the server's. Changing a device clock or editing the request body cannot alter what is charged. Verified against live orders in section 16.

9.2 — Authorisation in three independent layers

PROBLEM	Route-level protection alone is fragile: server actions are independently invocable, and a record addressed by ID can still be someone else's even when the route is legitimately reachable.
DECISION	Treat route protection as routing convenience and put the real enforcement next to the data.
IMPLEMENTATION	(1) <code>middleware.ts</code> matches the three area prefixes and redirects a mismatched role to its own dashboard. (2) Each role's <code>actions.ts</code> defines a private <code>require<Role>()</code> that re-reads the session and throws; all 13 actions call it first. (3) ID-addressed customer resources compare against the session — the invoice page calls <code>notFound()</code> unless <code>order.customerId === session.user.id</code> , and <code>cancelOrder</code> additionally requires the order to still be pending.
RESULT	No single bypass is sufficient. Read-only areas are enforced structurally too: the sales module contains no product-writing action at all, so a modified UI has nothing to call.

9.3 — Free items priced at zero instead of a discounted total

PROBLEM	"Buy 10 donuts, get 2 free" can be recorded either as money off the total or as extra units. Recording it as money off loses the information that specific items were gifted, and makes the invoice hard to reconcile against what was physically delivered.
DECISION	Append the free units as real order lines priced at zero.
IMPLEMENTATION	The engine returns <code>freeItems[]</code> alongside monetary discounts. <code>placeOrder</code> appends them as <code>OrderItem</code> rows with <code>price = 0</code> and <code>subtotal = 0</code> . The gifted unit is the cheapest qualifying item in the basket. Critically, paid and gifted units of the same product are summed into one map <i>before</i> the stock check, so ten purchased plus two free validates against twelve units — not ten.
RESULT	The invoice shows exactly what left the shop, and stock cannot go negative through a promotion. Visible in the live order in section 16.

9.4 — Two-tier image storage with an explicit failure mode

PROBLEM	A serverless host has a read-only filesystem, so uploads must go to object storage. But requiring a configured store just to run the project locally is friction, and a store accidentally created with private access rejects public uploads in a way that is easy to misdiagnose.
DECISION	Try object storage first, fall back to the local filesystem, and if both fail raise an error that names the actual cause and the fix.
IMPLEMENTATION	MIME allow-list and a 5 MB ceiling are checked first. The filename is generated server-side as <code><epoch>-<8 hex chars>.<ext></code> , so the user's filename never reaches storage. Deletion is guarded by the same pattern: a local file is only unlinked when its basename matches <code>/^\d{10,}-[0-9a-f]{8}\.w+\$/</code> .
RESULT	Local development works with no storage configuration, production failures are self-diagnosing, and the curated catalogue photographs cannot be deleted by editing a product.

9.5 — Login carries no role, and signup cannot choose one

PROBLEM	Any role information the client can express is information the client can forge.
DECISION	Remove role from the client surface entirely.
IMPLEMENTATION	The login form collects only an email and password; the role is read from the account record afterwards and used to choose the destination. The public signup endpoint hardcodes <code>CUSTOMER</code> and never reads a role from the request body — staff accounts can only be created by an existing administrator. Failed logins return one generic message so the form cannot be used to enumerate registered addresses.
RESULT	Privilege escalation through the public surface is not expressible.

9.6 — Deriving the canonical URL at runtime

PROBLEM	Authentication needs a canonical URL, but on a first deployment the domain is not known until after the deploy — so the correct value cannot be set beforehand, and logins misredirect until a second deploy.
DECISION	Derive it from the host platform's injected variables when it has not been set explicitly.
IMPLEMENTATION	At module load, if <code>NEXTAUTH_URL</code> is absent, the production deployment URL is used in production and the per-deployment URL in previews. An explicitly configured value always takes precedence.
RESULT	A first deployment authenticates correctly without a manual value and a second deploy; preview deployments work on their own generated domains.

9.7 — Product deletion that cannot corrupt history

PROBLEM	Removing a discontinued product must not break invoices that reference it.
DECISION	Let the database's referential integrity decide, and translate its refusal into a message a shop administrator can act on.
IMPLEMENTATION	<code>OrderItem</code> holds a restricting foreign key to <code>Product</code> . <code>deleteProduct</code> attempts the delete inside a <code>try/catch</code> ; on a foreign-key error it returns "This product can't be deleted because it appears in existing orders." Stored images are only cleaned up after the delete has actually succeeded.
RESULT	Historical invoices are structurally protected, and the failure is explained rather than surfaced as a stack trace.

SECTION 10

AI-assisted development

This project was built using an AI coding assistant as an implementation tool. The section below describes that working method and the division of responsibility, so the nature of the contribution is not ambiguous.

What the repository shows

EVIDENCE	WHAT IT INDICATES
A committed <code>.claude/launch.json</code>	An AI coding assistant (Claude Code) was configured and used within this repository.
All 8 commits authored by one person	Sole ownership of the repository, its history and every change merged into it.
Commit messages describing <i>intent</i> — e.g. "Make the three homepage offers actually apply", "Auto-derive NEXTAUTH_URL on Vercel", "Prepare project for Vercel deployment"	Work was directed goal-first in discrete, reviewable increments rather than generated in one pass.
Decision-rationale comments at the exact points of non-obvious choices	Reasoning was captured as the decisions were made — the storage fallback, the deletion filename guard, the cart hydration ordering, the mobile video autoplay fix, the omitted scrollbar.

Division of responsibility

OWNED BY THE DEVELOPER	DELEGATED TO THE ASSISTANT
Defining what the product is, who it serves, and which capabilities each role gets.	Translating an agreed capability into routes, components and queries.
Choosing the stack and the hosting model, and accepting their constraints.	Wiring the chosen libraries together correctly.
Specifying the business rules — the offer terms, who may cancel an order and when, what a read-only role means.	Expressing those rules as code, including edge cases such as summing gifted units into the stock check.
Running the application, finding defects in real use, and deciding whether a fix was acceptable.	Diagnosing and implementing the fix once the defect was reported.
Reviewing every change and committing it under their own name.	Producing the change for review.

How this shows in the code

The defects documented in section 15 are characteristic of this workflow: each was found by *using* the application — a cart that emptied on refresh, a video that would not autoplay on a phone, a hero subject cropped out of frame in portrait — and each was then fixed at the root cause rather than patched over. The explanatory comments left behind are the

record of that loop. Directing the work, reproducing the problems, and judging when a fix was correct were human responsibilities throughout; writing the lines was not.

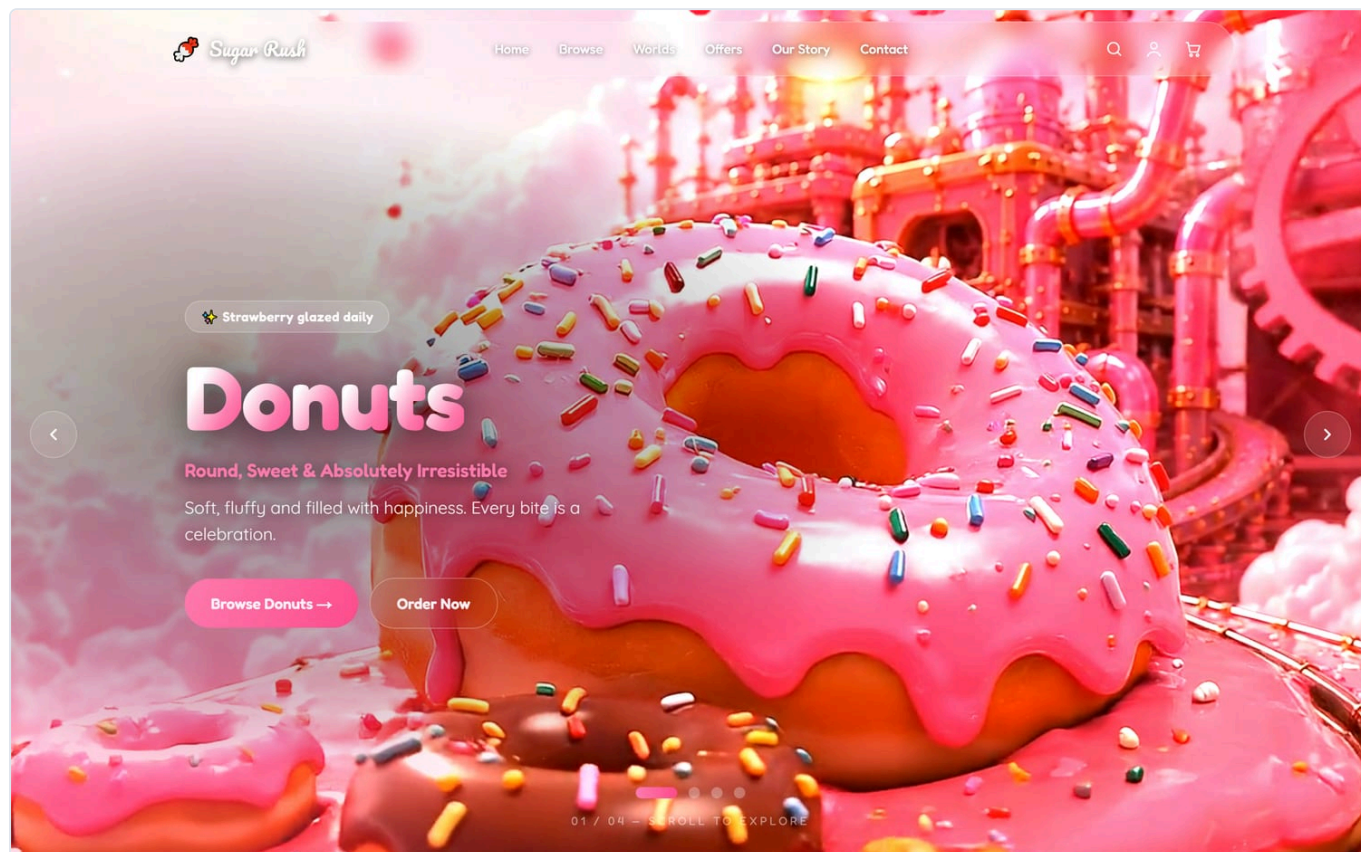
Stated plainly

The implementation was AI-assisted. The requirements, the architecture and product decisions, the validation of behaviour against real use, and responsibility for what shipped are the developer's. This report does not claim the code was typed by hand.

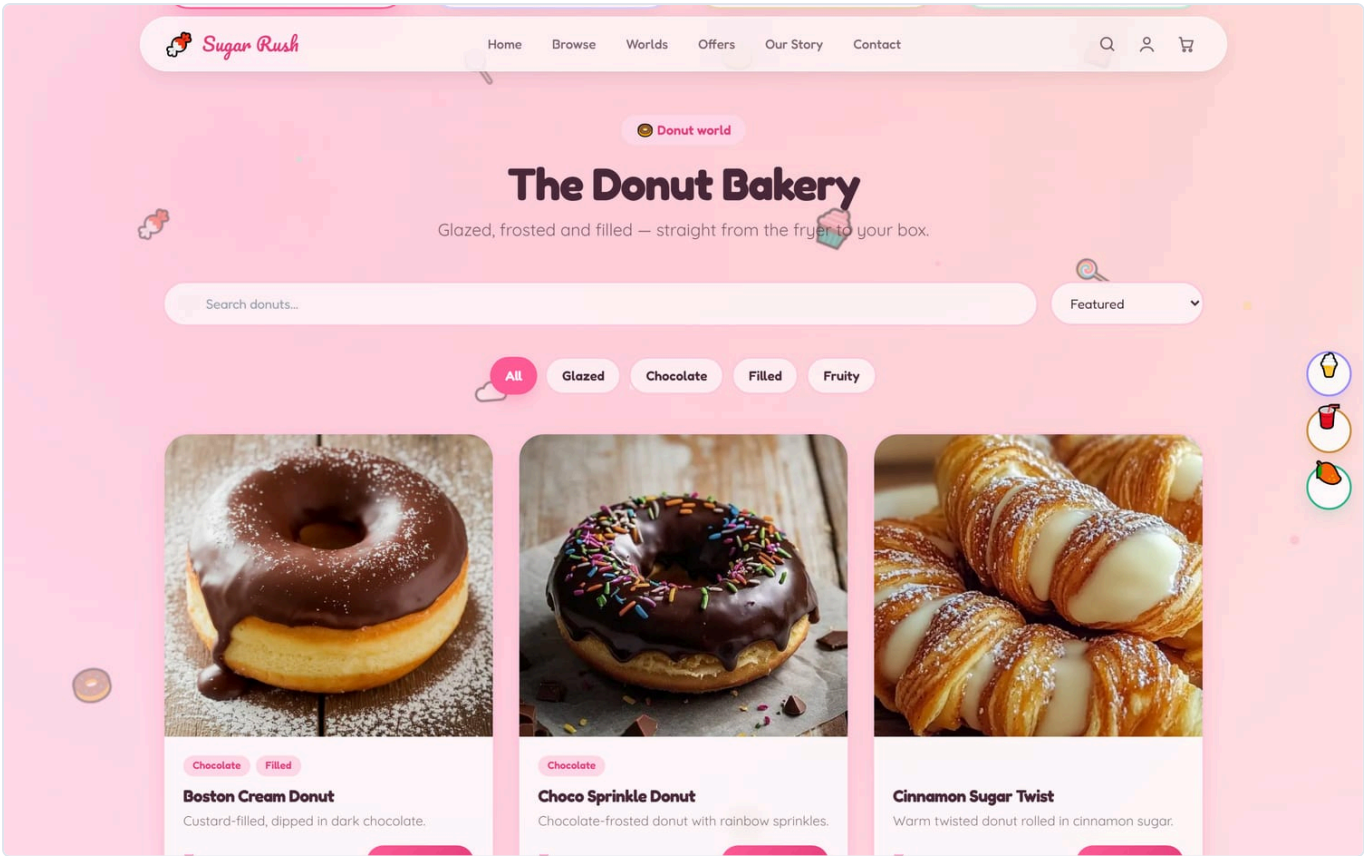
SECTION 11

Implementation — public storefront

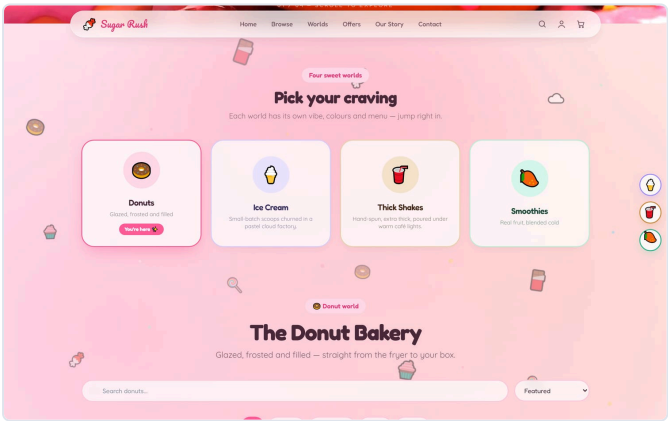
One continuous page. A full-bleed looping video introduces each of four dessert worlds, and the entire page below it — background gradients, accent colours, ambient particles, catalogue styling and filter chips — re-themes to match the active world.



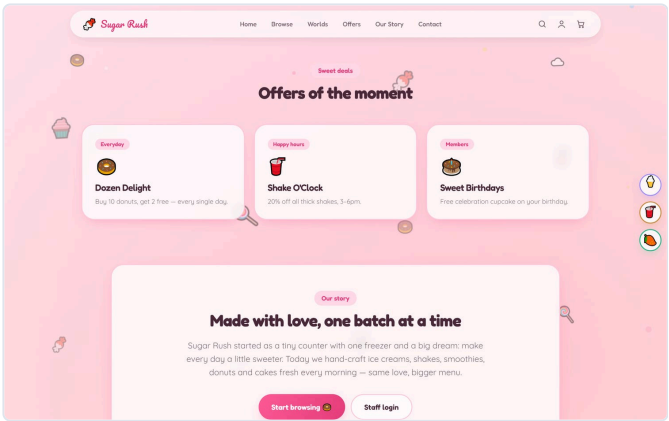
Hero / — the Donuts world. Each slide pairs an 8-second video loop with a poster image for first paint, a per-world title gradient, and a **01 / 04** progress indicator. Slides cross-fade rather than slide horizontally; the reason is documented in section 15.



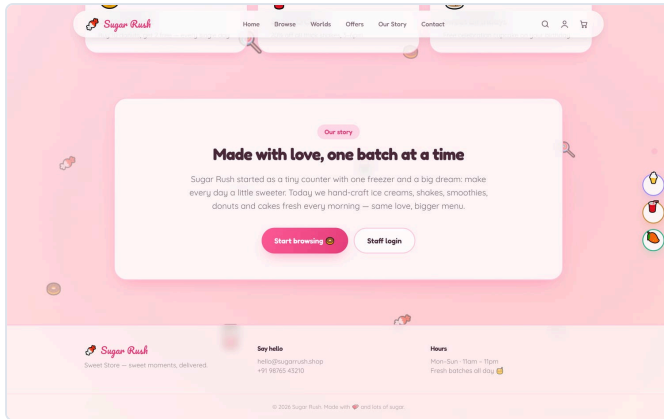
Catalogue `/#browse` — live products from PostgreSQL. The filter chips on each card ("Glazed", "Chocolate", "Filled") are derived at render time by running the active world's regular expressions over the product name and description, so adding a tag needs no schema change and no data migration. **Add to cart** works with no account.



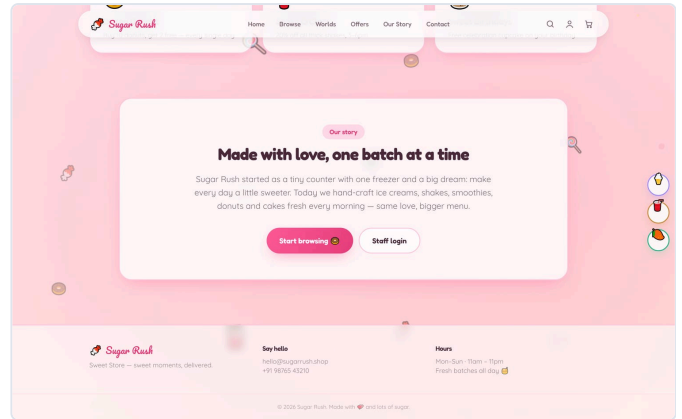
World selector `/#sweets` — switching re-themes everything below the hero.



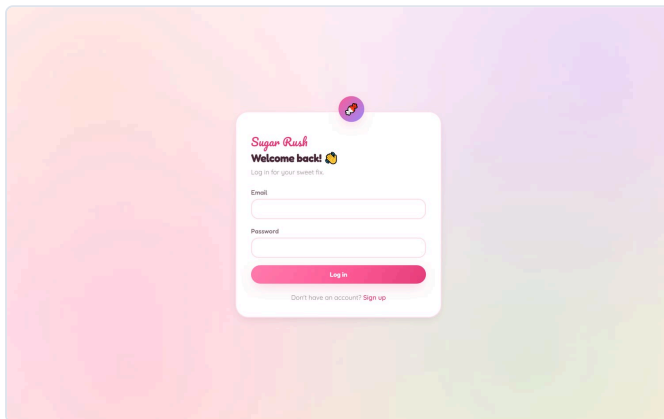
Offers `/#offers` — each card maps to an enforced rule, not marketing copy.



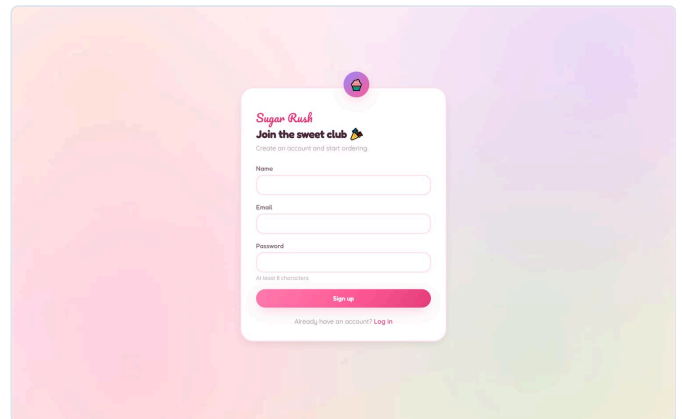
Story `/#story` — includes a staff login route alongside the shopping call to action.



Contact `/#visit` — details sourced from a single branding module.



Login `/login` — one form for all roles; the destination comes from the account.



Signup `/signup` — customer registration; role is fixed server-side.

Theming as configuration

Each world is one object in `lib/worlds.ts` carrying its accent colours, ambient glow, card border tint, a layered background gradient, an ingredient emoji set for the particle layer, a sprinkle palette, and its filter rules. Nothing about a world is hard-coded into a component, so adding a fifth world is a configuration change plus a video.

The catalogue image pipeline

Product photography was assembled by purpose-built tooling in `tools/` rather than collected by hand:

- `fetch-product-images.mjs` queries an openly-licensed image source restricted to permissive licences, rejects results whose titles suggest packaging, shopfronts, brand marks or people rather than the food itself, and requires the title to match the product it was asked for.
- `pick_by_color.py` addresses a failure the first script could not: a search for a kiwi smoothie returning a photograph that is the wrong colour. It downloads each candidate, measures the dominant hue and saturation of the centre of the frame, and accepts the first result inside the expected hue range.
- Attribution for every fetched image — title, creator, licence and source URL — is recorded in `public/products/CREDITS.json`; **22 images** are attributed there.

The recorded licences include three `by-nc-nd`, two `by-nd` and one `by-nc` entry. These are more restrictive than the permissive set the fetch script requests, so they appear to have been added by a different route. Before any commercial

use, those six images should be re-checked or replaced.

SECTION 12

Implementation — customer area



CUSTOMER

Order · track · cancel · invoice · support · account

Sugar Rush

Customer

Dashboard

Order sweets

My orders

Invoices

Support

Account settings

Order sweets

Add treats to your cart, then confirm delivery details.



Banana Coconut Gelato

Tropical banana-coconut gelato — summer in every scoop.

₹200.00

- 0 +



Belgian Waffle Stack

Crispy waffles with maple & fresh berries.

₹250.00

- 0 +



Berry Green Smoothie

Mixed berries, banana & spinach, blended smooth.

₹210.00

- 0 +



Blueberry Blast Shake

Hand-spun blueberry shake topped with cream.

₹230.00

- 0 +



Boston Cream Donut

Custard-filled, dipped in dark chocolate.

₹140.00

- 0 +



Choco Sprinkle Donut

Chocolate-frosted donut with rainbow sprinkles.

₹110.00

- 0 +



Chocolate Ice Cream

Rich chocolate scoops with a glossy fudge finish.

₹180.00

- 0 +



Chocolate Overload Shake

Triple chocolate, fudge drizzle & whipped cream.

₹250.00

- 0 +



Cinnamon Sugar Twist

Warm twisted donut rolled in cinnamon sugar.

₹100.00

- 0 +



Classic Glazed Donut

Fluffy ring donut with a sweet glaze.

₹90.00

- 0 +



Classic Tiramisu Cup

Coffee-soaked layers with mascarpone



Coconut Vanilla Ice Cream

Velvety classic vanilla with toasted

Your cart

No items yet.

Total ₹0.00

Delivery address

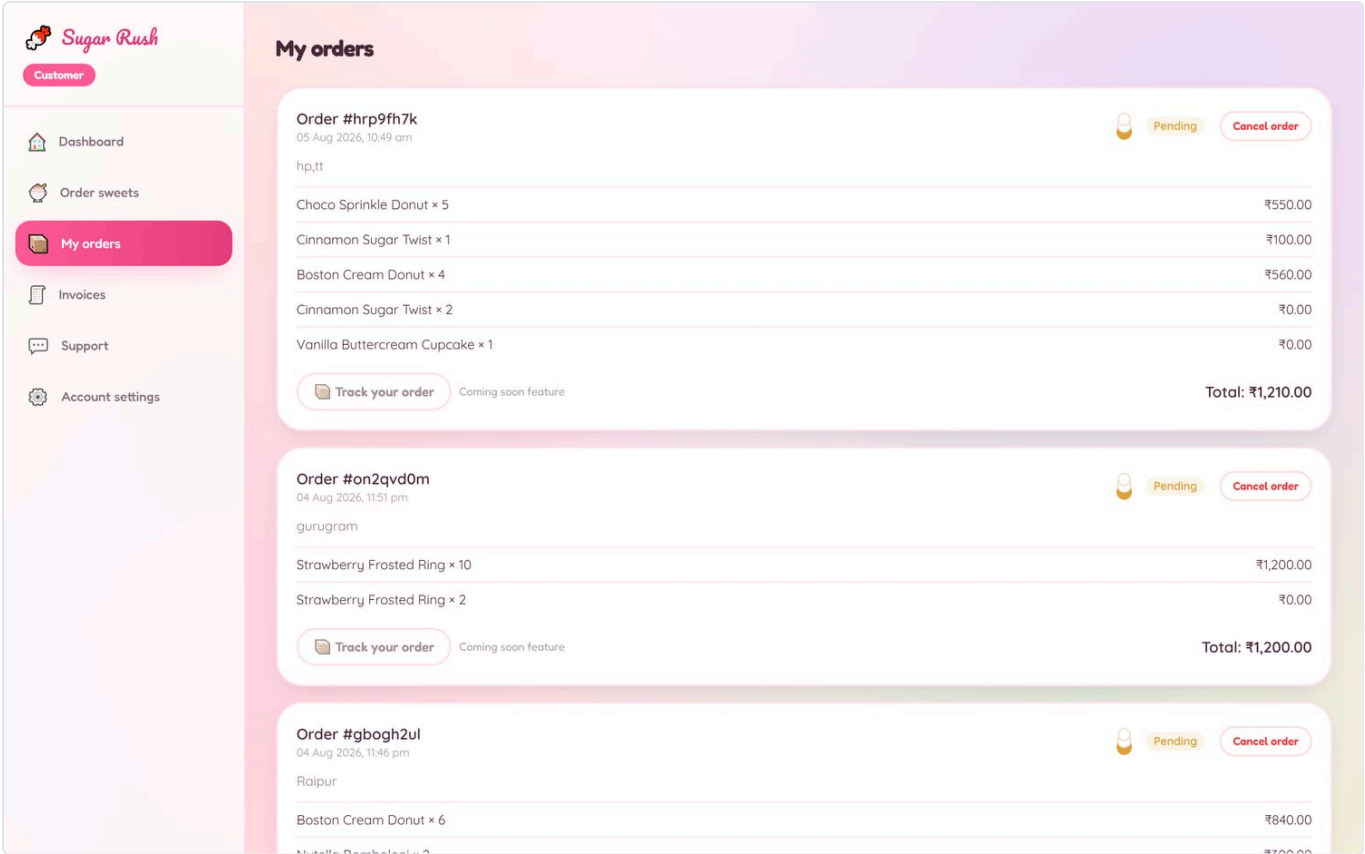
Payment method

Cash on Delivery

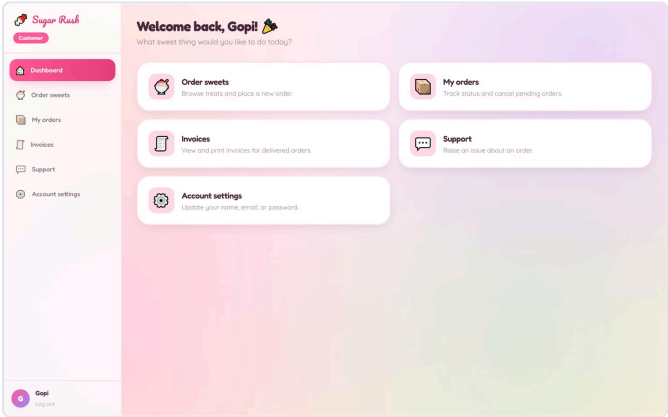
Place order

Order sweets `/customer/order` — the in-dashboard catalogue. Each product card carries a `-/+` stepper and the right-hand panel holds a running cart, delivery address and payment method. One page, one submitted order.

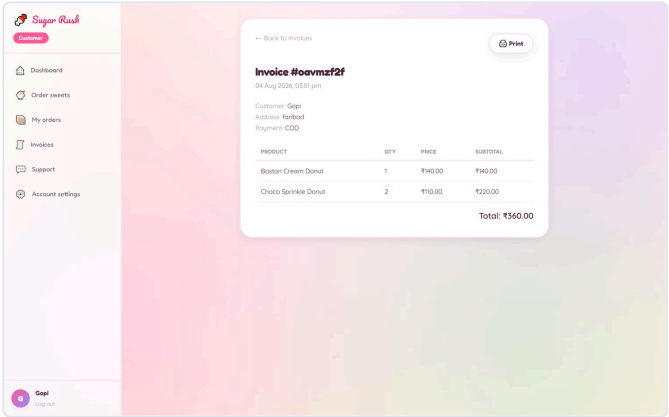
24 / 56



My orders `/customer/orders` — scoped to the signed-in customer. This screen is also the clearest evidence of the promotions engine working end to end. In order `#hrp9fh7k` the customer bought ten donuts across three products and received **Cinnamon Sugar Twist × 2 at ₹0.00** — the cheapest donut in the basket — plus a **Vanilla Buttercream Cupcake × 1 at ₹0.00** for the birthday offer. In `#on2qvd0m`, ten Strawberry Frosted Rings earned two more of the same at ₹0.00. The disabled "Track your order" control is a deliberate stub, labelled as such.



Dashboard `/customer/dashboard` — greeting from the session plus five action cards.



Invoice `/customer/invoices/[id]` — ownership-checked; print styles hide navigation.

Support `/customer/support` — stores a request against the session user.

Account `/customer/account` — name, email, optional password, and the birthday that enables the gift offer.

Ownership enforcement

Every customer-facing record is filtered by the session identifier rather than by a URL parameter. The order list queries `where: { customerId: session.user.id }`; the invoice detail page loads by ID and then calls `notFound()` unless the order belongs to the requester, so a guessed or shared identifier returns a 404 rather than another customer's data. `cancelOrder` applies the same check and additionally refuses if the order has already left the pending state.

SECTION 13

Implementation — sales area



SALES

Fulfilment desk — order queue, customer lookup, invoicing, stock reference



Sales

Dashboard

Manage orders

Customer info

Invoices

Inventory

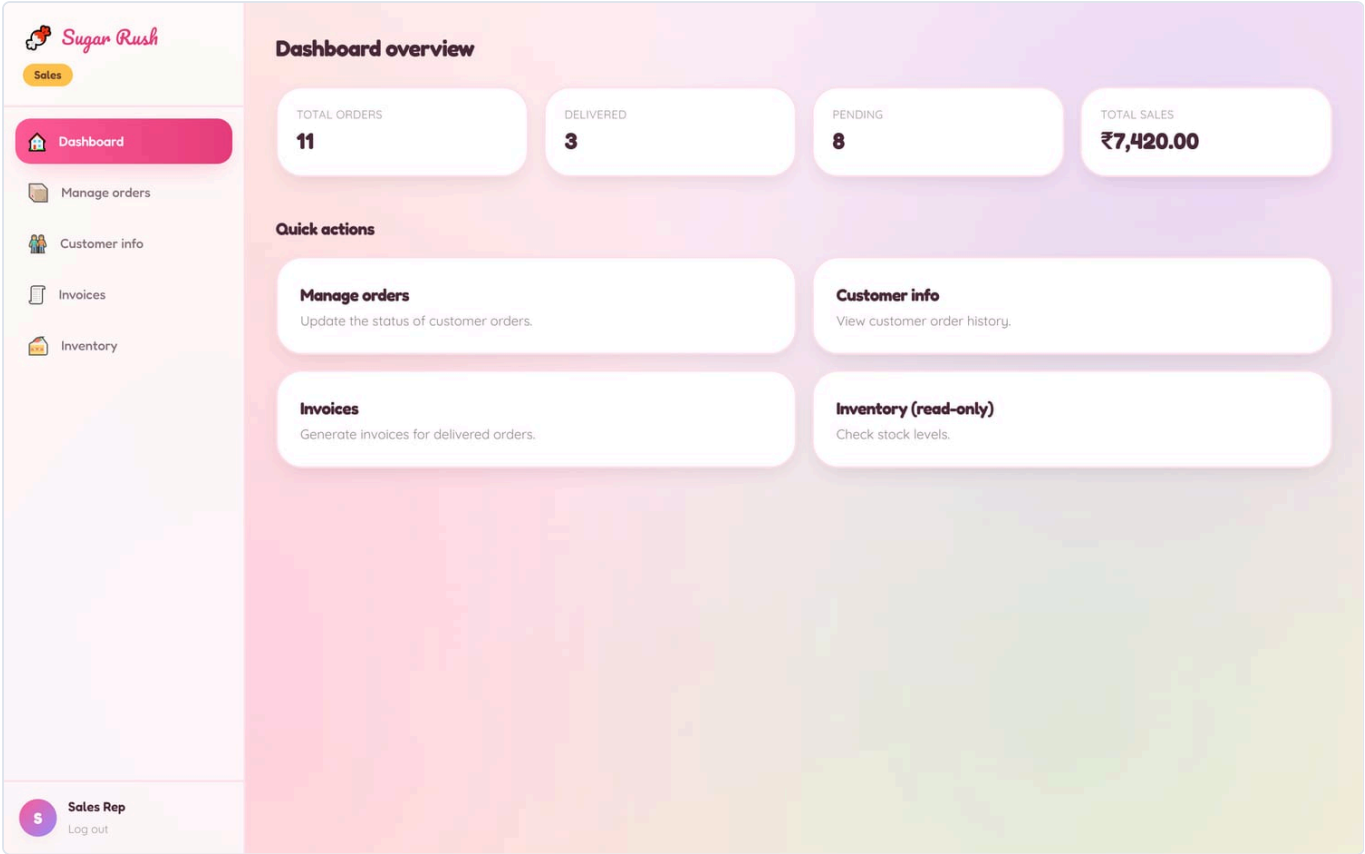
Sales Rep

Log out

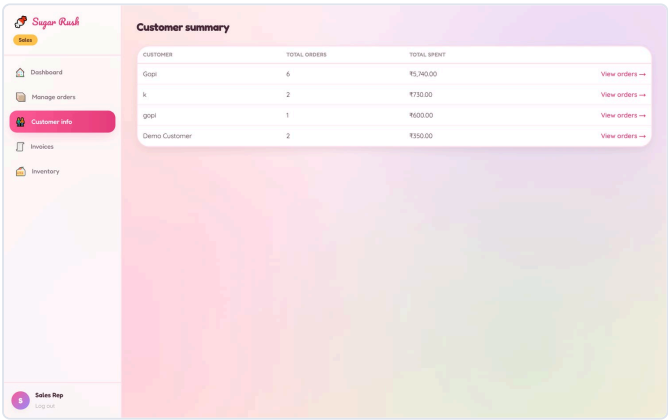
Manage orders

ID	CUSTOMER	TOTAL	ADDRESS	DATE	STATUS	UPDATE
#hrp9fh7k	Gopi	₹1,210.00	hp,tt	05 Aug 2026, 10:49 am	 Pending	Pending Update
#on2qvd0m	Gopi	₹1,200.00	gurugram	04 Aug 2026, 11:51 pm	 Pending	Pending Update
#gbogh2ul	Gopi	₹1,380.00	Raipur	04 Aug 2026, 11:46 pm	 Pending	Pending Update
#oavmzf2f	Gopi	₹360.00	faribad	04 Aug 2026, 03:51 pm	 Delivered	Delivered Update
#k21x9hko	k	₹350.00	mpp,v	20 Jul 2026, 04:38 pm	 Pending	Pending Update
#a539732l	k	₹380.00	hhharyan	20 Jul 2026, 04:30 pm	 Pending	Pending Update
#kodnpjjar	Demo Customer	₹210.00	rrraipir	20 Jul 2026, 04:27 pm	 Pending	Pending Update
#tntp3h0l	Demo Customer	₹140.00	12 Baker Street, Mumbai 400001	20 Jul 2026, 04:23 pm	 Delivered	Delivered Update
#fr3l7b9l	gopi	₹600.00	raipur	20 Jul 2026, 04:13 pm	 Pending	Pending Update
#4yqm446m	Gopi	₹1,000.00	UK, wales	07 Jul 2026, 02:15 pm	 Pending	Pending Update
#1yv0tkna	Gopi	₹590.00	raipur	07 Jul 2026, 02:05 pm	 Delivered	Delivered Update

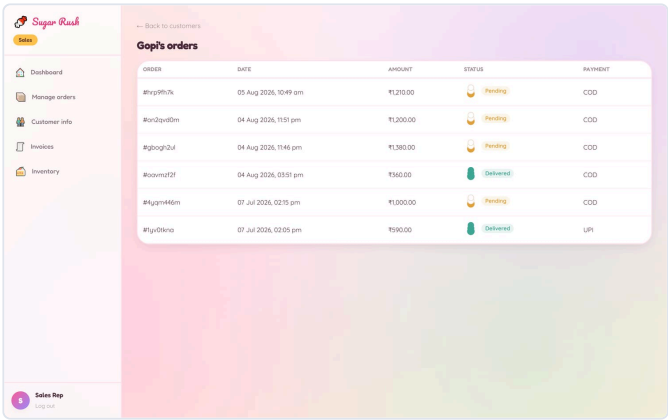
Manage orders `/sales/orders` — the fulfilment queue, newest first. The inline status control calls the single sales server action, which is constrained by zod to the three valid states and revalidates both this page and the dashboard so the KPI tiles never disagree with the table.



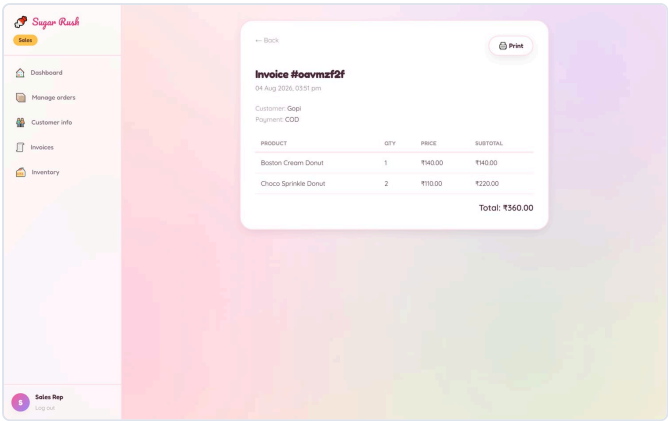
Dashboard `/sales/dashboard` — four KPI tiles resolved in a single `Promise.all` (total orders, delivered, pending, and a summed order value), plus quick actions.



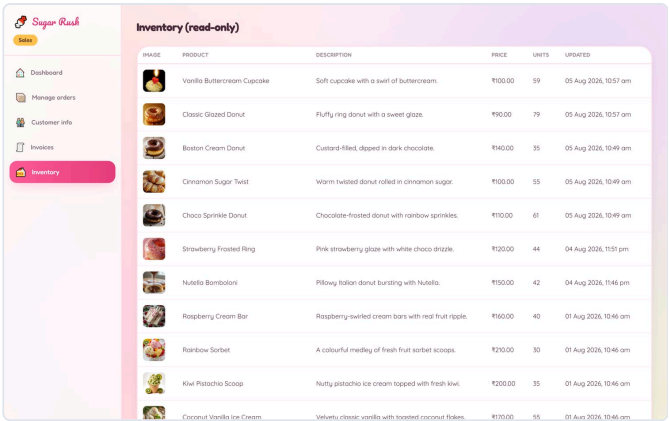
Customers `/sales/customers` — directory of accounts.



Customer detail `/sales/customers/[id]` — one customer's full order history.



Invoice `/sales/invoices/[id]` — same layout, staff-scoped: sales may view any order.



Inventory `/sales/inventory` — read-only, enforced by the absence of any write action.

Read-only as a structural property

The sales inventory screen has no edit controls, but more importantly `app/sales/actions.ts` contains exactly one exported action — `updateOrderStatus`. There is no product-mutating action within the sales role's reach, so modifying the client-side UI does not expose one. Restricting a role is a question of which actions exist, not which buttons are rendered.

SECTION 14

Implementation — admin area



Admin

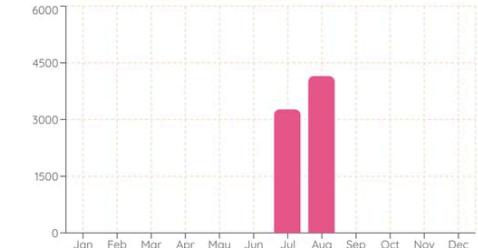
Manage users

Inventory

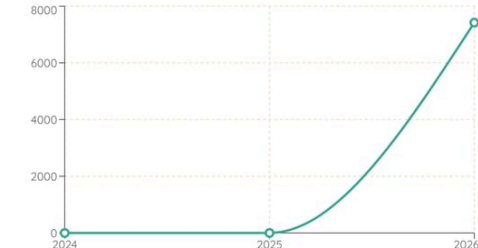
Sales report

Sales report

Monthly sales (2026)



Yearly sales



Order history

ID	CUSTOMER	TOTAL	ADDRESS	STATUS	PAYMENT	DATE
#hrp9fh7k	Gopi	₹1,210.00	hp,tt	Pending	COD	05 Aug 2026, 10:49 am
#on2qvd0m	Gopi	₹1,200.00	gurugram	Pending	COD	04 Aug 2026, 11:51 pm
#gbogh2ul	Gopi	₹1,380.00	Raipur	Pending	COD	04 Aug 2026, 11:46 pm
#oavmzf2f	Gopi	₹360.00	faribad	Delivered	COD	04 Aug 2026, 03:51 pm
#k21x9hko	k	₹350.00	mpp,v	Pending	COD	20 Jul 2026, 04:38 pm
#a53q737l	k	₹380.00	bbharan	Pending	COD	20 Jul 2026, 04:30 pm

Sales report `/admin/sales-report` — a monthly bar chart for the current year and a three-year line chart, both aggregated in the `Server` Component before rendering so no raw order data is shipped to the browser, with the complete order history below. The values shown are live records.

IMAGE	PRODUCT	UNITS	PRICE	UPDATED
	Vanilla Buttercream Cupcake	59	₹100.00	05 Aug 2026, 10:57 am
	Classic Glazed Donut	79	₹90.00	05 Aug 2026, 10:57 am
	Boston Cream Donut	35	₹140.00	05 Aug 2026, 10:49 am
	Cinnamon Sugar Twist	55	₹100.00	05 Aug 2026, 10:49 am
	Choco Sprinkle Donut	61	₹110.00	05 Aug 2026, 10:49 am
	Strawberry Frosted Ring	44	₹120.00	04 Aug 2026, 11:51 pm
	Nutella Bomboloni	42	₹150.00	04 Aug 2026, 11:46 pm
	Raspberry Cream Bar	40	₹160.00	01 Aug 2026, 10:46 am
	Rainbow Sorbet	30	₹210.00	01 Aug 2026, 10:46 am

Inventory `/admin/inventory` — thumbnail, name, stock, price and last-updated for every product, sorted by most recently changed, with per-row edit and delete. **Export CSV** calls an endpoint that performs its own authorisation check.

NAME	EMAIL	ROLE	CREATED
customer2	customer2@gmail.com	Customer	05 Aug 2026, 10:50 am
k	k@gmail.com	Customer	20 Jul 2026, 04:29 pm
king	king@gmail.com	Customer	20 Jul 2026, 04:29 pm
gopi	romano11610@gmail.com	Customer	20 Jul 2026, 04:19 pm
Gopi	gopipatel@gmail.com	Customer	07 Jul 2026, 02:59 pm
Demo Customer	customer@ecopacounter.com	Customer	07 Jul 2026, 01:59 pm
Sales Rep	sales@sugarrush.com	Sales	07 Jul 2026, 01:59 pm
Admin	admin@sugarrush.com	Admin	07 Jul 2026, 01:59 pm

Manage users `/admin/dashboard` — full CRUD with role assignment; self-deletion is refused.

Add product `/admin/inventory/new` — category constrained to the five valid values; image validated for type and size.

Two guards worth noting

- **Self-deletion is blocked.** `deleteUser` refuses when the target is the requesting administrator, which prevents the last admin from removing their own access and locking the organisation out of the system.
- **The export endpoint guards itself.** `/api/admin/export-inventory` is not covered by the middleware matcher, so it reads the session directly and returns **403** for any non-admin. Its CSV output escapes embedded quotes, commas and newlines rather than concatenating values naively.

SECTION 15

Challenges & solutions

Each of the following was a real defect or constraint encountered while building and using the application. All are documented in the source at the point of the fix.

15.1 — The cart emptied itself on every refresh

SYMPTOM	A guest cart built on the storefront was lost on page refresh, and — worse — lost across the login round-trip, which is exactly the journey the guest cart exists to support.
CAUSE	Two effects: one restoring the cart from <code>localStorage</code> after mount, one persisting it on change. A <code>useRef</code> was used to mark hydration as finished. Because a ref mutation does not schedule a render, the persist effect ran in the <i>same commit</i> as the hydrate effect, while <code>lines</code> was still the initial empty array — so it wrote <code>[]</code> straight over the saved cart before the restored value ever reached state.
FIX	Make the hydration flag <code>useState</code> rather than <code>useRef</code> , so setting it schedules a re-render and the persist effect only runs once the restored cart is genuinely in state.
RESULT	The cart survives refresh, navigation and sign-in. The reasoning is recorded as a comment so the ref is not reintroduced as an "optimisation".

15.2 — Hero videos would not autoplay on iOS

SYMPTOM	On desktop the hero played automatically; on iOS Safari it fell back to a tap-to-play control, which broke the intended first impression on the most common device class for a consumer storefront.
CAUSE	iOS only grants muted autoplay when the element carries the <code>muted</code> attribute. React sets the <code>muted</code> property on the DOM node but never renders the corresponding attribute, so the markup iOS inspected did not qualify.
FIX	In the element's ref callback, set <code>muted</code> and <code>defaultMuted</code> as properties <i>and</i> call <code>setAttribute("muted", "")</code> on the node, then invoke <code>play()</code> with the rejection swallowed. Legacy vendor-prefixed inline-playback attributes are also emitted for older WebViews.
RESULT	Autoplay works across desktop and mobile without a tap.

15.3 — The hero subject was cropped out of frame on phones

SYMPTOM	The hero videos are 16:9. On a portrait phone the subject was enlarged to fill a small part of the frame, and on one slide the shake glass was cropped off entirely.
CAUSE	A permanent overscan zoom existed to hide sub-pixel edge strips on desktop. In portrait, a 16:9 video already over-fills horizontally by a wide margin — no strip is possible — so that zoom only magnified an already-severe crop. Separately, subjects that sit off-centre were cropped away by a centred crop.
FIX	Apply the overscan as a <code>landscape:</code> -only CSS class so portrait skips it, and reduce the transition zoom amplitudes on portrait. Add an optional per-slide <code>focusPortrait</code> object-position; the thick-shake slide sets <code>75% 50%</code> because its glass sits at roughly 68% across the frame. The hero's minimum-height floor was also keyed to viewport <i>height</i> rather than width, so a landscape phone does not get a hero taller than its screen.
RESULT	The subject stays framed in portrait, landscape and on desktop.

15.4 — Slide transitions showed black bars at the edges

SYMPTOM	Mid-transition, dark strips appeared briefly down the sides of the hero.
CAUSE	Sliding the outgoing video layer sideways exposed the dark section background behind it for a few frames.
FIX	Replace the horizontal slide with a cross-fade combined with scale and blur, so no layer ever moves off its own bounds.
RESULT	Clean transitions with no exposed background.

15.5 — A scrollbar broke the full-bleed hero

SYMPTOM	A vertical strip of page background sat beside the hero video, so it was not truly edge to edge.
CAUSE	A classic (non-overlay) scrollbar reserves roughly 12px of layout width permanently.
FIX	Deliberately do <i>not</i> style scrollbar width, leaving the browser's overlay scrollbar in place, and set <code>overflow-x: clip</code> on the root elements. The decision is recorded in the stylesheet so it is not "fixed" later by adding custom scrollbar styling.
RESULT	The hero occupies the full viewport width.

15.6 — Sourced product photographs were often the wrong subject or colour

SYMPTOM	Automated image search returned packaging shots, shopfronts, brand marks and people instead of food — and sometimes the right subject in visibly the wrong colour.
CAUSE	Text relevance alone is a weak signal for whether a photograph depicts a specific prepared dish.
FIX	Two filters. A title-based reject list removes packaging, retail and human subjects and requires a positive keyword match for the requested product. Then a colour check downloads each candidate, measures the dominant hue and saturation of the centre of the frame, and accepts the first result inside the expected hue range. Licence filtering and attribution capture run alongside both.
RESULT	A coherent catalogue of 34 images with attribution recorded for 22 of them, assembled reproducibly rather than by hand.

15.7 — A first deployment could not authenticate	
SYMPTOM	Logins misredirected on the first deployment because the canonical URL was not yet known when the environment was configured.
CAUSE	An ordering problem in the platform, not a code defect: the domain is assigned by the deploy that needs it.
FIX	Derive the value at module load from the host's injected variables, preferring the stable production domain in production and the per-deployment domain in previews, while always deferring to an explicit setting.
RESULT	First deployments and preview deployments both authenticate without manual intervention. Committed separately as "Auto-derive NEXTAUTH_URL on Vercel".

SECTION 16

Testing & verification

There is no automated test suite

The project contains no test runner, no test files and no CI configuration. Everything below is either a compiler/build guarantee or a manual verification performed against the running application. This is a genuine gap and is listed again in section 21.

Automated checks that do exist

CHECK	RESULT	WHAT IT COVERS
<code>tsc --noEmit</code> TypeScript strict mode	Pass 0 errors	All 71 source files. <code>strict: true</code> is set, so implicit <code>any</code> , unchecked null access and unsound assignments are compile errors across the whole project.
<code>next build</code> Production build	Pass 27 routes	Compiles every route, prerenders the static ones, and traces server dependencies. It also confirms the build has no database dependency — it completes without reaching PostgreSQL.
Prisma schema validation	Pass	Runs as part of <code>prisma generate</code> in both <code>build</code> and <code>postinstall</code> .

Build output

```

Route (app)                                Size      First Load JS
├ o /                                       97 kB      206 kB
├ o /_not-found                             139 B      87.5 kB
├ f /admin/dashboard                       1.81 kB    89.2 kB
├ f /admin/inventory                       865 B      100 kB
├ f /admin/sales-report                    101 kB     188 kB
├ f /api/admin/export-inventory             0 B         0 B
├ o /checkout                             5.31 kB    114 kB
├ f /customer/order                       2.52 kB     95 kB
├ o /login                                 1.67 kB    106 kB
└ ... 18 further routes
+ First Load JS shared by all              87.4 kB
└ f Middleware                             49.4 kB

o (Static)  prerendered as static content    - 5 routes
f (Dynamic) server-rendered on demand        - 22 routes

```

The two heaviest bundles are the storefront (206 kB, which carries the animation libraries) and the sales report (188 kB, which carries the charting library). Both are justified by what those pages do; no other route exceeds 114 kB.

Manual verification against live data

The application was exercised end to end against its production PostgreSQL database while preparing this report. All twenty-five screenshots in sections 11–14 are from that session.

SCENARIO	RESULT	EVIDENCE
Authentication for all three roles	Pass	Admin, sales and customer accounts each authenticated and were routed to the correct dashboard by role.
Role-based route protection	Pass	Requesting a protected route without a session returns the login page rather than content.
Multi-buy promotion	Pass	Live order <code>#hrp9fh7k</code> : $5 + 1 + 4 = 10$ donuts across three products; two units of the cheapest (Cinnamon Sugar Twist, ₹100) were added at ₹0.00. Paid lines $550 + 100 + 560 = ₹1,210$, matching the stored total exactly.
Multi-buy, single product	Pass	Live order <code>#on2qvd0m</code> : $10 \times$ Strawberry Frosted Ring at ₹1,200, plus 2 of the same at ₹0.00. Total ₹1,200 .
Birthday gift promotion	Pass	Order <code>#hrp9fh7k</code> also carries Vanilla Buttercream Cupcake $\times 1$ at ₹0.00, applied because the order date matched the customer's stored birthday.
Gifted units recorded as order lines	Pass	All gifted items appear as distinct ₹0.00 lines in both the order history and the invoice.
Admin CSV export authorisation	Pass	Returns inventory for an admin session; the handler checks the session independently of middleware.
Sales reporting aggregation	Pass	Monthly and yearly charts rendered from 11 live orders; order history table matches.
Invoice ownership scoping	Pass	The customer invoice route resolves only for the owning session; the sales route resolves for staff.

What the promotion results demonstrate

These are not synthetic test fixtures — they are records created through the live ordering flow and read back from the database weeks later. They confirm the rule selection (cheapest qualifying item), the arithmetic, the zero-priced line representation, and the combination of two independent promotions on a single order.

SECTION 17

Security, reliability & data integrity

Security measures

AREA	IMPLEMENTATION
Injection	All database access goes through Prisma's parameterised client. No raw SQL and no query-string concatenation exists anywhere in the codebase.
Authentication	bcrypt at 10 salt rounds, applied consistently at signup, admin user writes and self-service updates. Hashes are never returned to the client. Failed logins yield one generic message, so the form cannot enumerate registered addresses.
Session	Stateless JWT with an 8-hour lifetime, carrying only the user id and role.
Authorisation	Three independent layers (section 9.2). Read-only roles are enforced by the absence of write actions, not by hidden buttons.
Privilege escalation	The public signup endpoint hardcodes the customer role and never reads one from the request. There is no role input anywhere on the public surface.
Direct object references	Every ID-addressed customer resource is verified against the session before rendering; mismatches return 404.
Cross-site request forgery	All mutations are Server Actions, which Next.js restricts to same-origin POST-style invocation. No state-changing GET route exists.
Open redirect	The post-login callback parameter is accepted only when it starts with <code>/</code> and not <code>//</code> — the second test is what rejects protocol-relative off-site URLs — and is honoured only for customer sessions.
File upload	MIME allow-list, 5 MB ceiling, server-generated random filename, storage in an object store rather than an executable web root.
Input validation	8 zod schemas with explicit bounds — quantity ≤ 50 per line, address 5–500 characters, message 5–2000, price strictly positive, stock non-negative, category and payment method closed enums.
CSV output	Quotes doubled, and any field containing a comma, quote or newline wrapped, so an injected product name cannot break the file structure.
Secrets	Environment variables only. <code>.env</code> git-ignored; <code>.env.example</code> documents shape without values. The admin password utility hardcodes nothing — omit the argument and it generates a random password and prints it once — which is why it is safe to keep committed.

Data integrity

- **Monetary precision.** Every money column is `Decimal(10,2)`. No currency value is ever held as a float.
- **Atomic ordering.** Order creation, order-item creation and every stock decrement occur inside one `prisma.$transaction`, so a partial order cannot exist.
- **Stock cannot go negative via promotions.** Paid and gifted units are summed per product before validation.

- **Historical accuracy.** Customer name and unit price are snapshotted onto the order, so invoices do not change when a user renames themselves or a product is re-priced.
- **Referential safety.** Products referenced by existing orders cannot be deleted; the refusal is surfaced as a readable message. Order items cascade with their order.
- **Idempotent seeding.** The seed upserts with an empty update clause, so re-running it adds missing rows without overwriting data — including without resetting a password that has since been changed.

Reliability

- **Database outage.** The storefront catches its query failure and renders with an empty catalogue, degrading to a marketing page instead of returning a server error.
- **Serverless connection handling.** A global Prisma singleton prevents client proliferation across function invocations; a pooled connection endpoint is specified for production.
- **Non-blocking cleanup.** Failed image deletion is swallowed, so housekeeping can never fail a user's request.
- **Non-blocking gifts.** An out-of-stock promotional item is skipped silently rather than rejecting a paid order.
- **Build independence.** No dynamic page is executed at build time, so deployment succeeds before the schema has been pushed.

SECTION 18

Deployment & usage

Local setup

```
# 1 – install
npm install
cp .env.example .env

# 2 – point DATABASE_URL and DIRECT_URL at a PostgreSQL instance,
#     then create the schema and seed demo data
npx prisma db push
npm run db:seed

# 3 – generate an auth secret into NEXTAUTH_SECRET
openssl rand -base64 32

# 4 – run
npm run dev           # http://localhost:3000
```

Object storage is optional locally: without a token, uploads fall back to writing into `public/products/`. The seed creates one account per role and populates 29 products across the five categories.

Hosted deployment

STEP	ACTION
1	Push to a Git remote. <code>.env</code> is ignored, so no secret is committed.
2	Collect both database endpoints: the pooled one for <code>DATABASE_URL</code> , the direct one for <code>DIRECT_URL</code> . Using the pooled endpoint is what prevents connection exhaustion under serverless concurrency.
3	Import the repository. The framework is detected automatically; default build settings apply because <code>build</code> already runs <code>prisma generate</code> .
4	Set the five environment variables. Create the Blob store with public read access — a private store rejects the upload and could not be rendered in an <code></code> in any case.
5	Deploy, then run <code>npx prisma db push</code> once against the production database to create the tables.

Operational notes

- Seeded accounts use a placeholder password documented in the repository. Rotate or remove them before exposing the application publicly.
- `scripts/reset-admin.ts` rotates or creates an administrator password; called without a password argument it generates a random one and prints it once.
- The project uses `prisma db push` rather than migrations, so schema changes are applied directly and are not versioned.
- The repository was developed on macOS. On another platform, run `npx prisma generate` (or a clean `npm install`) so the correct native query engine is produced.

SECTION 19

Role & contribution

Sole developer and owner. All eight commits in the repository are authored by me; there are no other contributors.

AREA	CONTRIBUTION
Product definition	Defined the three roles and the capabilities each receives, the promotion terms, and the rules governing who may change an order and when.
Architecture	Chose the stack and the serverless hosting model, and the consequences that follow: JWT sessions rather than a session table, object storage rather than filesystem uploads, pooled connections, a build with no database dependency.
Data modelling	Specified the five-model schema, including the deliberate denormalisation that keeps historical invoices accurate and the decimal precision for all monetary values.
Security model	Established the three-layer authorisation approach and the principle that the server re-derives every value that matters, rather than trusting client input.
Implementation direction	Drove the build in reviewable increments with an AI coding assistant, reviewed every change, and committed all of it under my own name. Section 10 describes this workflow and its division of responsibility.
Debugging & validation	Found the defects in section 15 by using the application on real devices — the emptying cart, the mobile autoplay failure, the portrait cropping, the transition artefacts — reproduced each, and verified the fixes.
Content pipeline	Specified and validated the licence-aware, colour-checked image sourcing tooling, and the attribution record it produces.
Deployment	Prepared and documented the hosted deployment path, including resolving the first-deploy authentication ordering problem.

Commit history

COMMIT	DATE	MESSAGE
a838418	2026-08-01	Initial commit
05eb659	2026-08-02	Prepare project for Vercel deployment
d5d7049	2026-08-03	Auto-derive NEXTAUTH_URL on Vercel
59c1f36	2026-08-03	Rename project to 'Sugar Rush' and add screenshots
4325403 , a6eaa27 , d9ba64f	2026-08-04	Repository consolidation and merge
9e6d464	2026-08-05	Make the three homepage offers actually apply

The final commit is the one that converted the storefront's advertised promotions into the enforced engine described in section 9.1.

SECTION 20

Results & verified metrics

Every figure below was measured from the repository, the build output, or the live database while preparing this report.

Codebase

71

TS / TSX FILES

5,502

LINES OF CODE

29

CLIENT COMPONENTS

13

SERVER ACTIONS

5

DATA MODELS

8

VALIDATION SCHEMAS

27

BUILD ROUTES

0

TYPE ERRORS

Measured results

METRIC	VALUE	SOURCE
TypeScript strict type check	0 errors	<code>tsc --noEmit</code>
Production build	Succeeds	<code>next build</code>
Routes compiled	27	Build output — 23 pages, 3 API handlers, 1 not-found
Prerendered static / dynamic	5 / 22	Build output
Shared First Load JS	87.4 kB	Build output
Heaviest route (storefront)	206 kB	Build output
Middleware bundle	49.4 kB	Build output
Server actions with a role guard	13 / 13	Source inspection
Action inputs with schema validation	13 / 13	Source inspection
Raw SQL statements	0	Source inspection
Automated tests	0	No test runner or test files present

Live application data at time of capture

MEASURE	VALUE	SOURCE
Products in catalogue	31	Admin CSV export
Total units in stock	1,253	Admin CSV export
Orders on record	11	Admin sales report
Promotions verified against live orders	3 of 3	Section 16

MEASURE	VALUE	SOURCE
Screens captured from the running app	25	Sections 11–14

Assets

ASSET CLASS	COUNT / SIZE	NOTES
Product photographs	34 · 7.9 MB	22 have attribution recorded in <code>CREDITS.json</code>
Hero videos + posters	4 + 4 · 15 MB	One 8-second loop per dessert world
Seeded products	29	6 each across four worlds, 5 desserts

No performance benchmark, load test, accessibility audit or Lighthouse score has been run, so none is quoted. The bundle sizes above are build measurements, not runtime performance claims.

SECTION 21

Limitations

Known gaps in the current state of the project, stated plainly. None is hidden elsewhere in this report.

Incomplete or absent functionality

GAP	DETAIL & IMPACT
No payment processing	<code>paymentMethod</code> records an intent only. No gateway is integrated and no payment is captured, authorised or verified. The system is not transactionally complete for real money.
Support requests are write-only	The customer form stores a <code>SupportRequest</code> correctly, but no admin or sales screen reads them. A customer receives no acknowledgement and staff have no inbox — the feature is only half a loop.
Order tracking is a stub	A disabled control labelled "Coming soon" appears on the order history. Nothing is implemented behind it. It is honestly labelled, but it is not a feature.
Cancellation does not restock	<code>cancelOrder</code> changes the status but does not increment <code>availableUnits</code> back. Stock removed at order time stays removed, so repeated cancellations progressively understate real availability.
No password reset	There is no self-service recovery. A customer who forgets their password requires an administrator to edit the account; staff recovery relies on the command-line utility.
No email or notification	No order confirmation, status-change notice or support acknowledgement is sent. Customers must return to the site to learn anything.
No automated tests	No test runner, test files or CI pipeline. Correctness currently rests on the type checker, the build, and manual verification. The promotions engine — pure, deterministic and the one component carrying financial logic — is the clearest omission.

Known inconsistencies

ISSUE	DETAIL
The in-dashboard order page does not preview offers	The storefront cart drawer previews promotions live, but <code>/customer/order</code> displays a plain price × quantity total. Because the server still applies promotions on submission, a customer ordering through the dashboard can be charged <i>less</i> than the total shown. The direction is favourable, but the two surfaces disagree and should be unified.
Six seeded image paths do not resolve	<code>chocolate-fudge.jpg</code> , <code>mango-delight.jpg</code> , <code>rainbow-sorbet.jpg</code> , <code>salted-caramel-crunch.jpg</code> , <code>strawberry-swirl.jpg</code> and <code>vanilla-bean.png</code> are referenced by the seed but absent from <code>public/products/</code> . Those cards fall back to a category emoji tile — graceful, but the references are dangling.
Six catalogue images carry restrictive licences	<code>CREDITS.json</code> records three <code>by-nc-nd</code> , two <code>by-nd</code> and one <code>by-nc</code> entry, which are outside the permissive set the fetch tooling requests. These must be re-checked or replaced before any commercial use.

ISSUE	DETAIL
Project name and brand differ	The package is named <code>scoop-counter</code> while the product is Sugar Rush, and the seeded demo accounts use a matching legacy domain. Cosmetic, but inconsistent.

Scale and operational constraints

- **No pagination.** Order, invoice, customer and inventory tables render every row. Adequate at the current volume of 11 orders and 31 products; it will require pagination or virtualisation well before either reaches the thousands.
- **No schema migration history.** The workflow is `prisma db push`, so schema changes are neither versioned nor reviewable, and there is no rollback path.
- **Single-currency, single-locale.** Prices are formatted as Indian rupees and promotion windows are evaluated in a single fixed timezone.
- **No rate limiting.** Neither login nor signup is throttled, so credential-stuffing is unmitigated at the application layer.
- **No observability.** There is no structured logging, error reporting or uptime monitoring.

SECTION 22

Future improvements

Prioritised by what the current state of the project most needs, rather than by novelty.

PRIORITY	IMPROVEMENT	RATIONALE & APPROACH
1	Unit-test the promotions engine	It is pure, deterministic, takes an injected context, and carries the only financial logic in the system — the cheapest test to write and the most expensive to omit. Cover: multi-buy thresholds and multiples, cheapest-item selection across mixed baskets, the happy-hour boundary hours, birthday matching across timezone edges, and combined promotions on one order. A lightweight runner plus roughly twenty cases would cover the whole module.
2	Restock on cancellation	A correctness defect, not a missing feature: stock silently drifts downward. Fix by incrementing <code>availableUnits</code> for every line inside the same transaction that sets the status, in both the customer and sales cancellation paths.
3	Unify offer preview across both ordering surfaces	Have <code>/customer/order</code> use the same <code>getOfferContext()</code> plus <code>computeOffers()</code> pairing the storefront cart already uses, so the displayed total always matches the charged total.
4	Close the support loop	Add a staff-facing list of <code>SupportRequest</code> records with a resolved/unresolved state. The data is already being captured, so this is a read surface plus one status column.
5	Transactional email	Order confirmation and status-change notices, which in turn unlock token-based password reset — the missing piece that currently forces administrator intervention.
6	Payment integration	Introduce a gateway and treat an order as provisional until payment is confirmed, rather than recording a payment intent as though it were settled.
7	Adopt Prisma migrations	Replace <code>db push</code> with a versioned migration history so schema changes become reviewable and reversible before the dataset is large enough to make mistakes costly.
8	Rate-limit authentication	Throttle login and signup by address and IP to blunt credential stuffing and automated account creation.
9	Paginate the staff tables	Apply cursor pagination to orders, invoices, customers and inventory before volume makes full-table rendering a problem.
10	Resolve the asset gaps	Replace the six restrictively-licensed images, supply or re-point the six dangling seed paths, and align the package name with the product name.
11	Implement order tracking	Replace the stub with real fulfilment milestones, which pairs naturally with the notification work in item 5.
12	Add observability	Error reporting and structured logging around the server actions, so production failures are visible rather than inferred.

SECTION 23

Conclusion

Sugar Rush delivers a complete, working ordering platform for a dessert retailer, covering the full path from anonymous browsing through checkout to fulfilment, invoicing and reporting, with three properly separated permission boundaries. It builds cleanly, type-checks cleanly under strict mode, and has been verified against live data.

The parts I consider the strongest engineering in the project are the ones where a requirement forced a real design choice rather than a mechanical one: a promotions engine defined once and trusted only from the server, so a live preview and an authoritative total cannot diverge; authorisation applied in three independent layers, with read-only roles enforced by which actions exist rather than which buttons render; and deliberate snapshotting in the data model so an invoice issued today still reads correctly after prices and names have changed. The defects documented in section 15 — the cart that emptied itself, the video that would not autoplay on a phone, the subject cropped out of frame in portrait — were each traced to a root cause and fixed there, with the reasoning recorded in the code.

The project is equally clear about what it is not. There is no payment processing, no notification of any kind, and no automated test suite; support requests are captured but never read; cancelling an order does not return its stock. These are listed in section 21 and prioritised in section 22, and the first three items there — testing the promotions engine, fixing the restock defect, and unifying the two offer previews — would materially raise the system's correctness rather than its feature count.

The implementation was AI-assisted, as section 10 sets out. The requirements, the architecture, the business rules, the validation against real use, and the responsibility for what shipped are mine.

SECTION 24

Master Prompt

A complete specification for rebuilding this project from an empty directory. Section 24.1 is the authoritative version, written in Claude's structured format. Section 24.2 gives the same specification adapted to five other prompt structures.

How to use it

Copy the whole of 24.1 into a coding assistant working in an empty folder. Afterwards you must still supply the five environment variables, run `npx prisma db push` and `npm run db:seed`, and add your own hero videos and product photographs — the prompt defines the media contract but cannot supply the media.

24.1 — Master Prompt (Claude structure)

▼ COPY FROM HERE

<role>

You are a senior full-stack engineer specialising in Next.js 14, TypeScript, Prisma and secure application architecture. You write production-quality code with strict typing, explicit validation and defence-in-depth authorisation, and you comment the reasoning behind non-obvious decisions rather than restating what the code does.

</role>

<context>

I am building "Sugar Rush", an online ordering platform for a dessert shop that sells ice cream, thick shakes, smoothies, donuts and baked desserts.

It serves three audiences from one codebase:

- VISITORS browse a themed catalogue and build a cart with no account. An account is required only at checkout.
- CUSTOMERS place orders, track and cancel them, view and print invoices, raise support requests, and manage their account.
- SALES staff work the fulfilment queue: view all orders, change order status, look up customer history, issue invoices, and read stock.
- ADMINISTRATORS manage users and roles, maintain the product catalogue including image uploads, export inventory as CSV, and review sales reporting.

It must deploy to serverless hosting (Vercel free tier) with managed PostgreSQL (Neon) and object storage for images. There is no separate API server and no long-running process.

The project directory is empty. Build everything.

</context>

<instructions>

1. PROJECT SETUP

Create a Next.js 14 App Router project in TypeScript with strict mode enabled and the path alias "@/*" mapped to the project root.

package.json name "sugar-rush", private, with scripts:

```
dev      → next dev
build    → prisma generate && next build
start    → next start
postinstall → prisma generate
db:push  → prisma db push
db:seed  → tsx prisma/seed.ts
db:studio → prisma studio
```

Dependencies: next 14.2.x, react 18.3.x, react-dom 18.3.x, @prisma/client ^5.20, next-auth ^4.24, bcryptjs ^2.4, zod ^3.23, @vercel/blob ^0.27, recharts ^2.12, framer-motion ^12, gsap ^3, lenis ^1.3.

Dev dependencies: typescript ^5.6, prisma ^5.20, tailwindcss ^3.4, postcss, autoprefixer, tsx ^4.19, @types/node, @types/react, @types/react-dom, @types/bcryptjs.

next.config.js must allow remote images from the blob storage hostname pattern.

2. DATA MODEL – prisma/schema.prisma

PostgreSQL datasource with BOTH `url` from DATABASE_URL and `directUrl` from DIRECT_URL.

```
enum Role { ADMIN SALES CUSTOMER }
```

```
enum OrderStatus { PENDING DELIVERED CANCELLED }
```

User

```
id      String  @id @default(cuid())
name    String
email   String  @unique
password String           // bcrypt hash
role    Role
birthday DateTime?       // optional; stored at UTC midnight, only month+day ever compared
createdAt DateTime @default(now())
orders  Order[]
supportRequests SupportRequest[]
```

Product

```
id          String  @id @default(cuid())
productName String
description  String
category    String  @default("desserts") // donuts | icecream | thickshakes | smoothies | desserts
price       Decimal @db.Decimal(10, 2)
availableUnits Int
imagePath    String?
lastUpdated  DateTime @updatedAt
orderItems   OrderItem[]
```

Order

```
id          String  @id @default(cuid())
customerId   String
customer     User    @relation(fields: [customerId], references: [id])
customerName String           // SNAPSHOT – invoices must survive a user rename
totalAmount  Decimal @db.Decimal(10, 2) // stored AFTER promotions
deliveryAddress String
orderDate    DateTime @default(now())
status       OrderStatus @default(PENDING)
paymentMethod String
items        OrderItem[]
```

OrderItem

```
id      String  @id @default(cuid())
orderId String
order   Order    @relation(fields: [orderId], references: [id], onDelete: Cascade)
productId String
product Product  @relation(fields: [productId], references: [id]) // restrict, NOT cascade
quantity Int
price    Decimal @db.Decimal(10, 2) // SNAPSHOT of unit price at order time
subtotal Decimal @db.Decimal(10, 2)
```

SupportRequest

```
id          String  @id @default(cuid())
customerId   String
customer     User    @relation(fields: [customerId], references: [id])
name         String
email        String
message      String
submittedAt  DateTime @default(now())
```

Never represent money as a float. The two snapshot fields above are deliberate – comment why.

3. AUTHENTICATION – lib/auth.ts, types/next-auth.d.ts, middleware.ts

NextAuth v4, Credentials provider only.

- authorize(): look the user up by LOWER-CASED, TRIMMED email, then bcrypt.compare. Return null on any failure. The UI

shows one generic "Incorrect email or password" so the form cannot enumerate registered addresses.

- jwt callback stamps id and role onto the token; session callback copies both onto session.user.
- session: { strategy: "jwt", maxAge: 60 * 60 * 8 }; pages: { signIn: "/login" }.
- At module load, if NEXTAUTH_URL is unset, derive it: use VERCEL_PROJECT_PRODUCTION_URL when VERCEL_ENV === "production", otherwise VERCEL_URL, and set `https://\${host}`. An explicit NEXTAUTH_URL always wins. Comment WHY: on a first deploy the domain does not exist until after the deploy.

types/next-auth.d.ts augments Session.user and JWT with `id: string` and `role: Role`.

middleware.ts uses withAuth with matcher ["/admin/:path*", "/sales/:path*", "/customer/:path*"]. Derive the required role from the path prefix. On mismatch REDIRECT the user to their own role's dashboard rather than showing a 403. authorized callback: !!token.

4. AUTHORISATION – three independent layers, all mandatory

- (a) Middleware gates page routes by role.
- (b) EVERY server action begins by calling its role folder's private requireAdmin() / requireSales() / requireCustomer(), which reads getSession and throws "Not authorized." on mismatch. Never rely on the middleware alone – server actions are independently invocable.
- (c) Ownership checks on every ID-addressed customer resource.

Additional rules:

- The login form must have NO role selector. Role comes from the account record. After sign-in, read the session, then route to /admin/dashboard, /sales/dashboard or /customer/dashboard.
- /login and /signup both accept a `callbackUrl` query parameter and must sanitise it identically:


```
const callbackUrl = raw.startsWith("/") && !raw.startsWith("//") ? raw : "";
```

 The `//` test rejects protocol-relative off-site URLs. Honour the callback only when the role is CUSTOMER.
- POST /api/signup is the ONLY public account-creation path and must HARDCODE role "CUSTOMER", never reading a role from the body. Validate with zod (name 1-120, valid email ≤200, password ≥8), return 409 on duplicate email, bcrypt-hash at 10 rounds before insert.

5. PROMOTIONS ENGINE – lib/offers.ts (the most important module; build it exactly)

One pure module is the single source of truth for three offers. It is imported by BOTH the cart UI (to preview a saving) and the server action that creates the order.

All time resolves in the store timezone "Asia/Kolkata" via Intl.DateTimeFormat – never the device clock.

Export:

```
computeOffers(lines, ctx)
  lines: { productId, name, category, unitPrice, qty }[]
  ctx: { happyHour: boolean, birthday: boolean }
  returns { subtotal, offers[], freeItems[], birthdayGift, discount, total }
```

```
isHappyHour(at?) – store-time hour >= 15 && < 18
isBirthdayToday(birthday, at?) – compares month+day only, in store time
```

Offer 1 – "Dozen Delight": buy 10 donuts, get 2 free. Sum the quantity of every line with category === "donuts". Each complete set of 10 grants 2 free units of the CHEAPEST donut in the basket. Grant these as free UNITS in freeItems[], NOT as money off – the invoice must show what was physically gifted.

Offer 2 – "Shake O'Clock": 20% off the `thickshakes` subtotal during happy hour.

Offer 3 – "Sweet Birthdays": when today's month+day matches User.birthday, attach one free "Vanilla Buttercream Cupcake". Resolve it server-side BY NAME because the cart does not carry the catalogue. If it is out of stock, SILENTLY SKIP the gift – never block a paid order over a missing freebie.

Security requirements – these are the point of the design:

- Expose a getOfferContext() server action so the client learns happyHour/birthday FROM THE SERVER. Changing the device clock must have no effect.
- placeOrder accepts ONLY { productId, quantity } per line. It re-reads every price and category from the database, re-reads the birthday from the User row, evaluates its own clock, and recomputes from scratch. Store the server's number in Order.totalAmount. Ignore anything the browser calculated.
- Sum paid units and gifted units of the same product into ONE map BEFORE the stock check, so 10 donuts + 2 free validates against 12 units, not 10.
- Create the order, its items and every stock decrement inside a single prisma.\$transaction.

6. SERVER ACTIONS

```
app/admin/actions.ts – createUser, updateUser, deleteUser, createProduct, updateProduct, deleteProduct
app/sales/actions.ts – updateOrderStatus (this file must contain NOTHING else, so no product-writing action is within the sales role's reach)
```

app/customer/actions.ts – getOfferContext, getMyBirthday, placeOrder, cancelOrder, submitSupportRequest, updateAccount

Rules:

- deleteUser must refuse when `userId === session.user.id`, so an admin cannot lock the organisation out.
- updateUser/createUser reject a duplicate email with a readable message.
- deleteProduct wraps the delete in try/catch and, on a foreign-key error, returns "This product can't be deleted because it appears in existing orders." Delete the stored image only AFTER the row delete succeeds.
- Product create/update/delete must revalidatePath("/admin/inventory") AND revalidatePath("/"), because the storefront catalogue is ISR-cached.
- updateOrderStatus is zod-constrained to PENDING | DELIVERED | CANCELLED and revalidates both /sales/orders and /sales/dashboard.
- cancelOrder must verify the order is BOTH owned by the session user AND still PENDING.
- Every action input is zod-validated with explicit bounds: quantity ≤ 50 per line, deliveryAddress 5-500, support message 5-2000, price strictly positive, availableUnits ≥ 0 , category and paymentMethod closed enums.

7. ROUTES

Public: / · /login · /signup · /checkout · custom not-found

Admin: /admin/dashboard · /admin/inventory · /admin/inventory/new · /admin/inventory/[id]/edit · /admin/sales-report

Sales: /sales/dashboard · /sales/orders · /sales/customers · /sales/customers/[id] · /sales/invoices · /sales/invoices/[id] · /sales/inventory

Customer: /customer/dashboard · /customer/order · /customer/orders · /customer/invoices · /customer/invoices/[id] · /customer/support · /customer/account

API: /api/auth/[...nextauth] · /api/signup · /api/admin/export-inventory

Each role folder has its own layout.tsx rendering a shared DashboardShell with that role's badge and navigation.

Screen specifics:

- /admin/dashboard – user CRUD with role assignment via a UserManager client component.
- /admin/inventory – table of thumbnail, name, units, price, lastUpdated sorted by lastUpdated desc, per-row Edit/Delete, plus "Add product" and "Export CSV".
- /admin/sales-report – Recharts monthly bar chart (current year, 12 buckets) and a 3-year line chart, both AGGREGATED IN THE SERVER COMPONENT before rendering, plus the full order-history table.
- /sales/dashboard – four KPI tiles (total orders, delivered, pending, summed totalAmount) resolved in one Promise.all.
- /sales/inventory – read-only stock view, no edit controls.
- /customer/order – in-dashboard catalogue where each product is a card with a +/- stepper, and a right-hand live cart with running total, delivery-address textarea, payment-method select (Cash on Delivery / UPI / Card) and Place order. ONE page, ONE submitted order – do not build a multi-step session cart. Use the same getOfferContext() + computeOffers() pairing as the storefront cart so the displayed total always matches the charged total.
- /customer/orders – own orders only, scoped by session id, with a Cancel button on pending orders.
- /customer/invoices/[id] – MUST call notFound() unless order.customerId === session.user.id.
- Invoice pages are printable: hide navigation chrome with ``print:hidden``.

/api/admin/export-inventory – CSV of ID, Product Name, Available Units, Price, Last Updated. It is NOT covered by the middleware matcher, so it MUST guard itself: read getSession and return 403 unless role === "ADMIN". CSV-escape every value (double embedded quotes; wrap any field containing a comma, quote or newline). Send Content-Disposition: attachment with a dated filename.

8. STOREFRONT – app/page.tsx + components/home/*

A single long, immersive page, NOT a plain product grid.

app/page.tsx is a Server Component that fetches all products, maps them to plain objects and passes them to a client HomeExperience. Set `export const revalidate = 60`. Wrap the Prisma call in try/catch and fall back to an EMPTY catalogue, so a database outage degrades the page to a brochure instead of a 500.

lib/worlds.ts defines FOUR dessert worlds – donuts, icecream, thickshakes, smoothies – matching Product.category values. Each carries: name, emoji, browseTitle, browseSub, chip, accent, accentDark, accentSoft, glow (rgba), ink, cardBorder, a multi-layer `backdrop` gradient stack, an `ingredients` emoji array for the particle layer, a `sprinkles` colour array, and `tags` – filter chips defined as { label, match: RegExp } which are run over the product name + description at render time, so tags need no database column.

lib/homeData.ts defines four HERO_SLIDES, each with an 8-second looping mp4, a poster jpg, title, tagline, description, two CTAs, accentFrom/accentTo, a glow colour and a chip label. Slides auto-advance every 8000 ms with a 1000 ms cross-fade. Support an optional `focusPortrait` object-position string per slide.

Sections in order: full-bleed video hero with glass navbar and an "01 / 04 – scroll to explore" indicator → world selector (#sweets) → catalogue (#browse) → offers (#offers) → story (#story) → contact/footer (#visit). Selecting a world must re-theme EVERYTHING below the hero.

Hero implementation details – these are real fixes, implement them:

- iOS grants muted autoplay only when the element carries the `muted` ATTRIBUTE. React sets the property but never renders the attribute. In the ref callback set `el.muted`, `el.defaultMuted` AND `el.setAttribute("muted","")`, then call `play().catch(()=>{})`. Also emit `webkit-playsinline` and `x5-playsinline` for older WebViews.
- Apply the overscan zoom that hides sub-pixel edge strips as a `landscape`-only class. In portrait a 16:9 video already over-fills horizontally, so the extra zoom would only magnify an already-tight crop.
- In portrait, apply the slide's focusPortrait as `objectPosition` so an off-centre subject is not cropped away.
- Transition between slides by CROSS-FADE with scale and blur, never a horizontal slide – sliding the layer exposes the dark section background at the edges for a few frames, which reads as black bars.
- Key the hero's minimum-height floor to viewport HEIGHT, not width, so a landscape phone does not get a hero taller than its screen.

GUEST CART – `components/CartContext.tsx`: persist cart lines to `localStorage` so a visitor can shop with no account.

CRITICAL: the "has hydrated" flag must be `useState`, NOT `useRef`. With a ref, the persist effect runs in the same commit as the hydrate effect while ``lines`` is still the initial `[]`, overwriting the saved cart on every refresh. Comment this so it is not "optimised" back to a ref.

`/checkout` is the only step requiring login: redirect unauthenticated visitors to `/login?callbackUrl=%2Fcheckout` and restore the cart afterwards; forward the callback through the signup → login hop too. If a STAFF account reaches checkout, show a "Customer account needed" card rather than a broken form.

9. IMAGE UPLOADS – `lib/blob.ts`

`uploadProductImage(file)`:

1. Reject unless `MIME ∈ {image/jpeg, image/png, image/webp, image/gif}`; reject if size > 5 MB. Throw readable errors.
2. Generate the filename SERVER-SIDE as ``${Date.now()}-${randomBytes(4).toString("hex")}.${ext}`` – never use the user's filename.
3. If `BLOB_READ_WRITE_TOKEN` exists, put() to blob storage with access "public" and return the URL.
4. If that throws – most often because the store was created PRIVATE, which rejects public uploads – `console.warn` and fall back to writing into `public/products/`, returning a site-relative path.
5. If that also fails, throw an error naming the real cause and the fix. Comment that the local fallback is a development convenience: a serverless production filesystem is read-only.

`deleteProductImage(url)`: for a blob URL call `del()`; for a local `/products/` path, ONLY unlink when the basename matches `/^d{10,}-[0-9a-f]{8}\.w+$/`. This guarantees an edit can never delete a curated/seeded catalogue photo. Swallow all cleanup errors – never fail a request on housekeeping.

10. DESIGN SYSTEM

`tailwind.config.ts` extends the theme with:

- colors: ink #4A2A3A, base #FFF4FA, surface #FFFFFF, border #FFE1EE, plus DEFAULT/dark/light triples for raspberry (#FF5C97 / #E63E7C / #FFD9E7), blueberry (#9B8CFF / #7A67F0 / #EAE4FF), pistachio (#42D2A8 / #28B389 / #D6F7EC), caramel (#FFC24D / #E89A1E / #FFF0CC), peach (#FF9E6D / #F07C43 / #FFE6D6), choco (#8A5A44 / #6B4130 / #EAD9CF), bubble (#5FC9F3 / #2FA9DC / #DAF2FC).
- fontFamily: display → Pacifico, heading → Fredoka, sans → Quicksand, all loaded via `next/font/google` as CSS variables.
- borderRadius x12 1.25rem, x13 1.75rem, and an organic `blob` radius; boxShadow candy, soft, pop.
- keyframes + animations: float, float-x, wiggle, pop, fade-up, spin-slow, marquee, sparkle, blobmorph.

`app/globals.css` defines an `@layer` components set: `.btn`, `.btn-primary` (pink gradient), `.btn-secondary`, `.btn-mint`, `.btn-lemon`, `.btn-danger`, `.card`, `.card-hover`, `.input`, `.label`, `.table-base`. Body gets a fixed multi-radial-gradient background. Set `overflow-x: clip` on `html/body` and do NOT style scrollbar width – a classic scrollbar reserves ~12px beside the full-bleed hero video. Comment this decision.

Components to build: `DashboardShell` (sidebar shell shared by all three roles), `ScoopStatus` (an ice-cream-scoop SVG status pill – 1 filled scoop amber for Pending, 3 filled teal for Delivered, a crossed circle for Cancelled), `SalesCharts`, `userManager`, `ProductForm`, `OrderBuilder`, `CartContext` / `CartBar` / `CartDrawer`, `OrderStatusUpdater`, `CancelOrderButton`, `DeleteProductButton`, `PrintButton`, `Providers`, and under `components/home/`: `HomeExperience`, `HeroSlider`, `GlassNavbar`, `ProductWorld`, `WorldBackdrop`, `WorldContext`, `WorldQuickNav`, `SmoothScroll`, `FloatingIngredients`, `CandyParticles`, `MagneticButton`.

Accessibility: honour prefers-reduced-motion in the `Lenis` smooth-scroll driver and the particle layers; lazy-mount particles with an in-view check; keep particle animation to `transform/opacity` only; use deterministic pseudo-random placement so layouts are stable between renders.

`lib/format.ts` – `formatCurrency` rendering ₹ with en-IN grouping and 2 decimals; `formatDate` in en-IN "02 Aug 2026, 03:51 pm" style.

`lib/brand.ts` – a single `BRAND` object (name, tagline, emoji, blurb, email, phone) so renaming the shop is a one-file change.

11. SEED & SCRIPTS

`prisma/seed.ts` must be `IDEMPOTENT` – `upsert` with an `EMPTY` update object, so re-running never overwrites data or resets a

changed password. Create one user per role, all with a clearly-placeholder password, and seed ~29 products across the five categories with realistic Indian rupee prices (donuts ₹90-150, ice cream ₹150-210, thick shakes ₹220-260, smoothies ₹180-210, desserts ₹100-250), each with a description and an imagePath under /products/. Only reference image files you actually create.

scripts/reset-admin.ts – usage `npx tsx scripts/reset-admin.ts <email> [newPassword]`. If the password argument is omitted, generate a random one with randomBytes and print it ONCE. Hardcode nothing, so the script is safe to keep committed.

12. ALSO PRODUCE

- .env.example documenting DATABASE_URL (pooled in production), DIRECT_URL, NEXTAUTH_SECRET, NEXTAUTH_URL and BLOB_READ_WRITE_TOKEN, with comments explaining the pooled-vs-direct distinction.
- README.md – stack, local setup, database setup, auth secret generation, blob setup (stressing the store must be PUBLIC access), how to run, and the project structure.
- DEPLOY.md – step-by-step hosted deployment: push to Git, collect both connection strings, import the project, set the five variables, deploy, then run `prisma db push` once against production.
- A custom app/not-found.tsx.

</instructions>

<constraints>

- No string-built SQL anywhere. Prisma parameterised queries only.
- Never trust a price, total, discount, role or ownership claim from the client.
- Every mutation is a Server Action (same-origin POST). No state-changing GET route such as ?delete=5.
- Validate every input with zod, with explicit maximum lengths.
- bcrypt at 10 rounds everywhere a password is written; never return a hash to the client.
- Money is always Decimal(10,2); never a float.
- Secrets live in environment variables only; .env is git-ignored.
- Ownership checks on every ID-addressed customer resource.
- Restrict a role by which server actions EXIST, not by which buttons render.
- The whole project must compile cleanly under TypeScript strict mode.
- Comment WHY a non-obvious decision was made, not what the code does – especially the storage fallback, the image-deletion filename guard, the offers engine's client/server trust split, the cart hydration ordering, the iOS autoplay attribute, and the omitted scrollbar styling.

</constraints>

<output_format>

Produce the complete working project as files on disk. Create every file needed for `npm install && npm run build` to succeed with no manual edits beyond supplying environment variable values.

Build in this order, so each stage is verifiable before the next depends on it:

1. Project scaffolding, config, Tailwind theme, globals.css
2. Prisma schema, lib/prisma.ts, seed script
3. lib/auth.ts, types augmentation, middleware.ts, login/signup pages, /api/signup
4. lib/offers.ts, lib/blob.ts, lib/worlds.ts, lib/homeData.ts, lib/brand.ts, lib/format.ts
5. Shared components (DashboardShell, ScoopStatus, forms, cart)
6. Customer area, then sales area, then admin area
7. Storefront (components/home/*) and /checkout
8. README.md, DEPLOY.md, .env.example

After each stage, state briefly what you built and what remains.

</output_format>

<final_check>

Before finishing, verify:

- `npx tsc --noEmit` reports zero errors under strict mode.
- `npm run build` succeeds without any database connection.
- All 13 server actions call their role guard as the first statement.
- No server action trusts a client-supplied price, total or role.
- placeOrder recomputes offers from database prices and the server clock, and sums paid + gifted units before the stock check.
- The customer invoice route returns notFound() for an order the session does not own.
- /api/admin/export-inventory returns 403 for a non-admin session.
- app/sales/actions.ts exports exactly one action.
- deleteUser refuses self-deletion; deleteProduct returns a readable message on a foreign-key error.
- The cart survives a refresh and a login round-trip.
- Both the storefront cart and /customer/order display the same total that placeOrder charges.

</final_check>

24.2 — The same specification in other prompt structures

The technical content does not change between models; only the framing does. Below, each structure is filled in with the framing appropriate to that model. Where a block says **[SPEC]**, insert the `<instructions>` and `<constraints>` bodies from 24.1 verbatim — that is the substance of the prompt and it is identical in every variant.

OpenAI structure

Role: Senior full-stack engineer building a production Next.js 14 + TypeScript + Prisma application from an empty directory. You own the architecture, the data model, and the security posture of what you produce.

Personality

Precise and decisive. Prefer explicit validation over convenience. Comment the reasoning behind non-obvious decisions, never restate what the code does. Do not ask for confirmation on choices the specification already settles.

Goal

A complete, working online ordering platform for a dessert shop — "Sugar Rush" — serving three roles (Admin, Sales, Customer) plus anonymous visitors, deployable to serverless hosting with managed PostgreSQL and object storage.

Success criteria

- `npx tsc --noEmit` reports zero errors under strict mode.
- `npm run build` succeeds with no database connection available.
- All 13 server actions call a role guard as their first statement.
- `placeOrder` recomputes promotions from database prices and the server clock, never from client input.
- Paid and gifted units are summed before the stock check.
- The customer invoice route returns `notFound()` for an order the session does not own.
- `/api/admin/export-inventory` returns 403 for a non-admin session.
- The guest cart survives a refresh and a login round-trip.

Constraints

[SPEC — the `<constraints>` block from 24.1]

Output

The complete project as files on disk, built in the eight-stage order given in 24.1. After each stage, one short paragraph on what was built and what remains.

Stop rules

If a requirement is genuinely ambiguous, choose the option that keeps authority on the server and note the choice — do not stop to ask. Stop only when every success criterion above is verifiably met.

Specification

[SPEC — the `<instructions>` block from 24.1]

GPT-4.1 structure

Role and Objective

Senior full-stack engineer. Build "Sugar Rush", a complete online ordering platform for a dessert shop, from an empty directory: Next.js 14 App Router, TypeScript strict, Prisma + PostgreSQL, NextAuth, deployed serverless.

Instructions

[SPEC — the `<instructions>` block from 24.1]

Sub-categories

- Data integrity: money is always `Decimal(10,2)`; snapshot `customerName` and unit price onto orders.
- Authorisation: three independent layers — middleware, per-action role guard, per-record ownership check.
- Client trust: the client sends identifiers and quantities only; the server re-derives every value that matters.
- Failure behaviour: fail soft on the inessential (out-of-stock gift, failed image cleanup, database outage on the storefront); fail loud and specifically on the essential (misconfigured image store).

Reasoning Steps

1. Model the data first and confirm the schema supports every workflow before writing UI.

2. Build authentication and the three authorisation layers before any protected screen.
3. Build lib/offers.ts as a pure function, then wire the server-authoritative context around it.
4. Build one role area end to end before starting the next.
5. Build the storefront last – it depends on the catalogue shape being settled.

Output Format

Complete project files on disk, in the eight-stage build order from 24.1, with a short status note after each stage.

Context

The directory is empty. Three roles (Admin, Sales, Customer) plus anonymous visitors. Serverless host with a read-only filesystem, managed PostgreSQL, and object storage for images.

Final instructions

[SPEC – the <final_check> block from 24.1]

Codex structure

Objective

Build "Sugar Rush", a complete Next.js 14 online ordering platform for a dessert shop, with three role-based areas (Admin, Sales, Customer), a public storefront with an anonymous cart, and a server-authoritative promotions engine.

Context

Next.js 14 App Router · TypeScript strict · Prisma + PostgreSQL · NextAuth (Credentials, JWT) · Tailwind · Recharts · Vercel Blob. Deployment target is serverless with a read-only filesystem, so uploads go to object storage and the build must not require a database connection.

Current State

Empty directory. No scaffolding, no dependencies, no schema.

Requirements

[SPEC – the <instructions> block from 24.1]

Constraints

[SPEC – the <constraints> block from 24.1]

Files and directories that matter:

- prisma/schema.prisma – five models, two enums, Decimal money, both url and directUrl
- lib/offers.ts – pure promotions engine, the single source of truth for all three offers
- lib/blob.ts – validated upload, object-store-then-local fallback, guarded delete
- lib/auth.ts, middleware.ts – authentication and layer 1 of authorisation
- app/{admin,sales,customer}/actions.ts – every action guarded by its own require<Role>()
- app/sales/actions.ts must export exactly one action

Acceptance Criteria

- `npx tsc --noEmit` → 0 errors under strict mode
- `npm run build` → succeeds with no database reachable
- All 13 server actions call a role guard first
- placeOrder ignores client-supplied prices and totals entirely
- Paid + gifted units summed before the stock check
- Customer invoice route 404s for an order the session does not own
- /api/admin/export-inventory returns 403 for a non-admin
- deleteUser refuses self-deletion
- Guest cart survives refresh and the login round-trip

Verification

- Run `npx tsc --noEmit` and `npm run build`; both must pass.
- Verify placeOrder with a basket of 10 donuts across three products: two units of the cheapest must be appended at price 0, and the stock check must validate against 12 units.
- Verify the invoice route with a foreign order id returns notFound().
- Confirm app/sales/actions.ts exports only updateOrderStatus.
- Confirm no file contains raw SQL.

Deliverable

A complete project where `npm install && npm run build` succeeds with no manual edits beyond supplying environment variable values, plus README.md, DEPLOY.md and .env.example.

Gemini structure

```

<role>
You are a senior full-stack engineer specialising in Next.js 14, TypeScript, Prisma and secure application architecture.
</role>

<instructions>
[SPEC – the <instructions> block from 24.1]
</instructions>

<constraints>
[SPEC – the <constraints> block from 24.1]
</constraints>

<output_format>
The complete working project as files on disk, built in the eight-stage order given in 24.1, with a brief status note
after each stage. `npm install && npm run build` must succeed with no manual edits beyond environment variable values.
</output_format>

<context>
"Sugar Rush" is an online ordering platform for a dessert shop selling ice cream, thick shakes, smoothies, donuts and
baked desserts. It serves anonymous visitors (browse and build a cart with no account), customers (order, track, cancel,
invoice, support, account), sales staff (fulfilment queue, customer lookup, invoicing, read-only stock) and
administrators (users and roles, catalogue, CSV export, sales reporting). The deployment target is serverless with a
read-only filesystem, managed PostgreSQL and object storage. The project directory is empty.
</context>

<task>
Build the entire application from scratch, exactly to the specification above.
</task>

<final_instruction>
The single most important property is that the server never trusts the client. Prices, totals, discounts, roles and
ownership are always re-derived server-side; the client sends identifiers and quantities only. If any instruction appears
to conflict with that principle, the principle wins. Before finishing, verify every item in the final-check list from
24.1.
</final_instruction>

```

Universal structure

```

ROLE
Senior full-stack engineer – Next.js 14, TypeScript, Prisma, secure application architecture.

OBJECTIVE
Build "Sugar Rush", a complete online ordering platform for a dessert shop, from an empty directory.

CONTEXT
Three role-based areas (Admin, Sales, Customer) plus a public storefront with an anonymous cart.
Serverless deployment: read-only filesystem, managed PostgreSQL, object storage for images.
No separate API server; no long-running process.

REQUIREMENTS
[SPEC – the <instructions> block from 24.1]

CONSTRAINTS
[SPEC – the <constraints> block from 24.1]

SUCCESS CRITERIA
- tsc --noEmit: 0 errors under strict mode
- next build: succeeds with no database connection
- 13/13 server actions guarded by a role check as their first statement
- 0 client-supplied prices, totals or roles trusted anywhere
- Promotions recomputed server-side from database prices and the server clock
- Paid + gifted units summed before every stock check

```

- Ownership verified on every ID-addressed customer resource
- Guest cart survives refresh and the login round-trip

EXAMPLES

Correct: `placeOrder` receives `[{ productId, quantity }]`, re-reads each price from the database, recomputes offers from the server clock, writes its own total.

Incorrect: `placeOrder` receives `[{ productId, quantity, price, discount, total }]` and writes the total it was given.

Correct: `app/sales/actions.ts` exports only `updateOrderStatus`, so no product write is reachable by a sales session.

Incorrect: a shared actions module exporting product writes, with the sales UI simply not rendering the buttons.

OUTPUT FORMAT

Complete project files on disk, in the eight-stage build order from 24.1, with a short status note after each stage.

VERIFICATION / STOP CONDITIONS

Run the full final-check list from 24.1. Stop only when every item passes. If a requirement is ambiguous, choose the option that keeps authority on the server and note the decision.